
지도학습 1강

— Supervised Learning Overview —

Image Classification

airplane



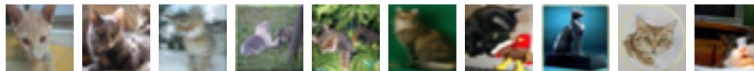
automobile



bird



cat



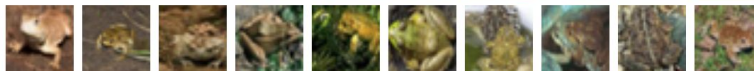
deer



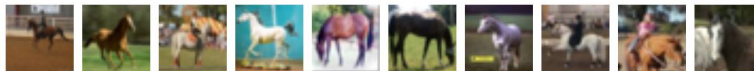
dog



frog



horse



ship



truck



Text Classification



이동진 평론가   5년 전

기생충

영화 · 2019

★ 4.5

상승과 하강으로 명징하게 직조해낸 신랄하면서 처연한 계급 우화.

Next Word Prediction

The cat sat on the _____

- table (30%)
- chair (35 %)
- mat (32%)
- ...

Translation

영어 - 감지됨 ▼



Korean ▼

My dog is cute



우리 강아지는 귀여워요

uli gang-ajineun gwiyeowoyo



[Open in Google Translate](#) • [Feedback](#)

Price Prediction



Supervised Learning

- Data: $x \in \mathcal{X}$ usually a vector
- Label: $y \in \mathcal{Y}$
- Dataset:
 $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots (x^{(n)}, y^{(n)})$

Image Classification



$$x \in \mathcal{X}$$

airplane

$$y \in \mathcal{Y}$$

airplane

automobile

bird

cat

deer

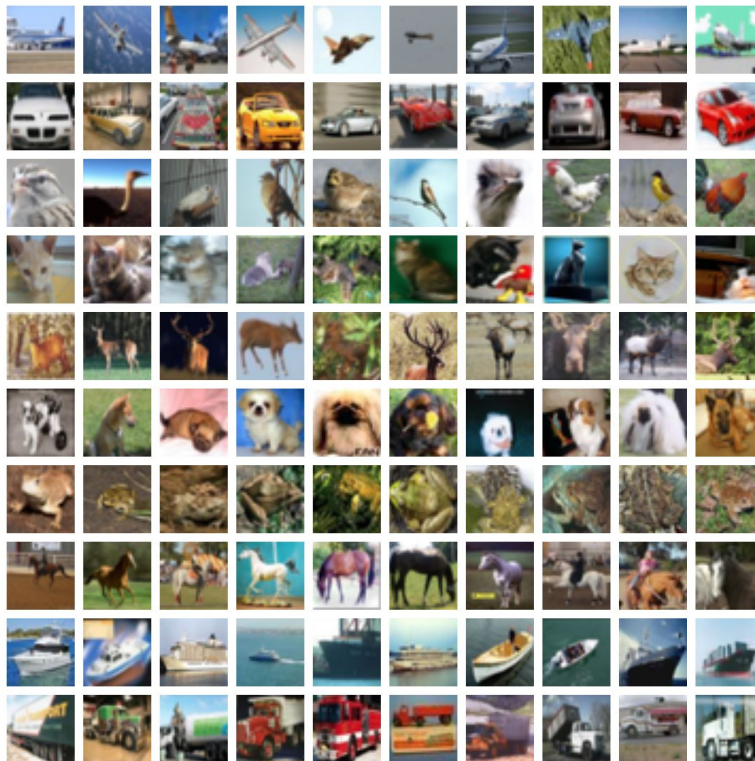
dog

frog

horse

ship

truck



Text Classification



이동진 평론가   5년 전

기생충

영화 · 2019

★ 4.5

상승과 하강으로 명징하게 직조해낸 신랄하면서 처연한 계급 우화.

$x \in \mathcal{X}$

Positive
 $y \in \mathcal{Y}$

Next Word Prediction

The cat sat on the _____

$$x \in \mathcal{X}$$

$$\begin{array}{c} \text{mat} \\ y \in \mathcal{Y} \end{array}$$

Translation

영어 - 감지됨

↔

Korean

My dog is cute

×

우리 강아지는 귀여워요

$x \in \mathcal{X}$

$y \in \mathcal{Y}$

uli gang-ajineun gwiyeowoyo

[Open in Google Translate](#) • [Feedback](#)

Price Prediction



Before Machine Learning

- Rule-based Algorithms (based on expertise, knowledge)
 - What defines image '0'?
 - When stock prices goes up?
 - Cannot solve complicated tasks
- Machine Learning
 - Data based
 - Provide data and let algorithm decides

Supervised Learning

Machine Learning

Machine Learning is

"the field of study that gives computers the ability to learn without being explicitly programmed."



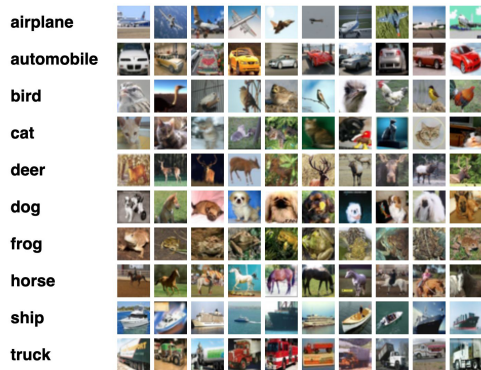
Arthur Lee Samuel

@photo from Forbes.com

Supervised Learning

- Classification

Discrete (finite) $y \in \mathcal{Y}$



영어 - 감지됨

My dog is cute



Korean

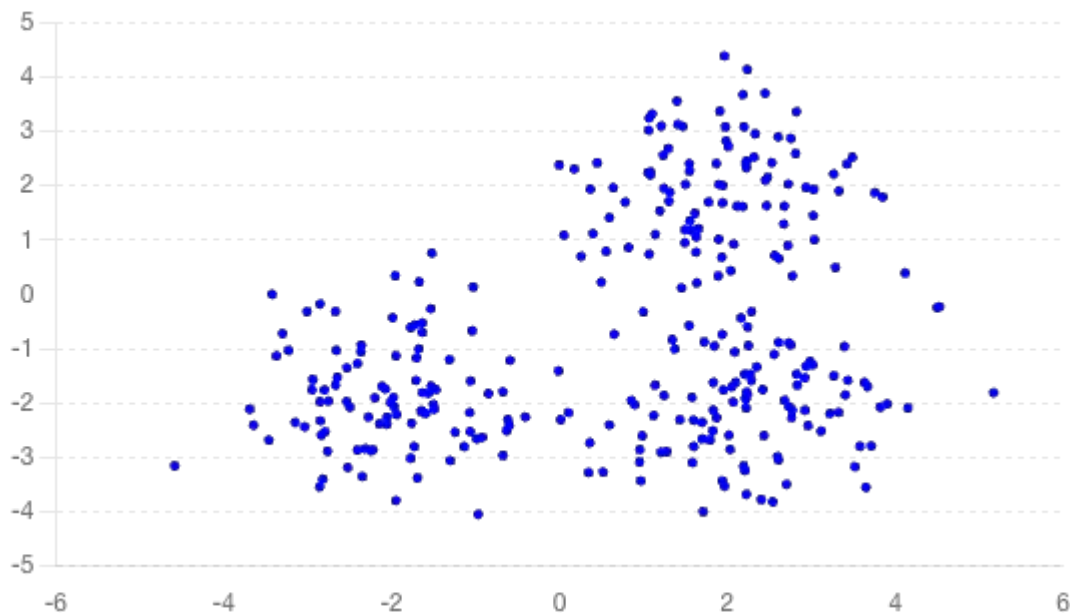
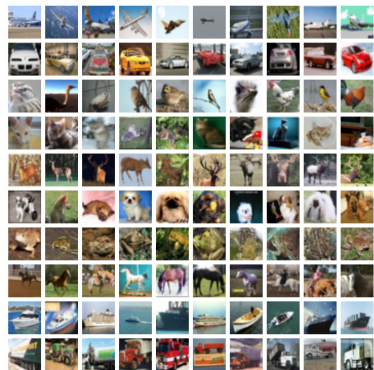
우리 강아지는 귀여워요
uli gang-ajineun gwiyeowoyo

- Regression

Continuous (real) $y \in \mathcal{Y}$



Unsupervised Learning



Supervised Learning: Setup

- True function: $f^\star$, where $f^\star(x^{(i)}) = y^{(i)}$
- Want to find

$$g(x) \approx f^\star(x)$$

- Function class: $\mathcal{G}$
- Goal:

$$g_\theta(x) \approx f^\star(x)$$

$$g_\theta \in \mathcal{G}$$

Find g_θ from function class that approximates the true function **well**

Approximate well?

- Similar function values for all input x [Infeasible]

$$g_{\theta}(x) \approx f^{\star}(x)$$

- Similar function values for given dataset

$$g_{\theta}(x^{(i)}) \approx f^{\star}(x^{(i)}) = y^{(i)}$$

Approximate well?

- Similar function value can be measured

$$g_{\theta}(x^{(i)}) \approx f^{\star}(x^{(i)}) = y^{(i)}$$

- Pointwise Loss
 - Can be mean-squared error, cross-entropy, etc.

$$\ell(g_{\theta}(x^{(i)}), y^{(i)})$$

- Loss

$$\mathcal{L}(\theta) = \sum_{i=1}^n \ell(g_{\theta}(x^{(i)}), y^{(i)})$$

Supervised Learning

- Given dataset (with label)

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots (x^{(n)}, y^{(n)})$$

- Set function class $\mathcal{G}$
- Set loss function ℓ
- Find $g_{\theta} \in \mathcal{G}$ that minimizes

$$\mathcal{L}(\theta) = \sum_{i=1}^n \ell(g_{\theta}(x^{(i)}), y^{(i)})$$

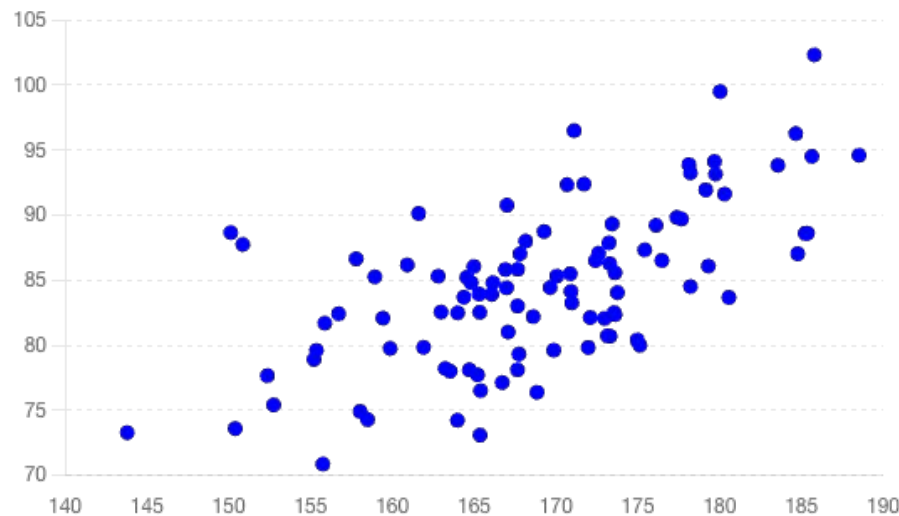
Linear regression example

- heights vs. weights

$$x \in \mathcal{X} = \mathbb{R}, y \in \mathcal{Y} = \mathbb{R}$$

- It has positive correlation
- Function Class (Model)

$$\mathcal{G} = \{g_{a,b}(x) = ax + b\}$$

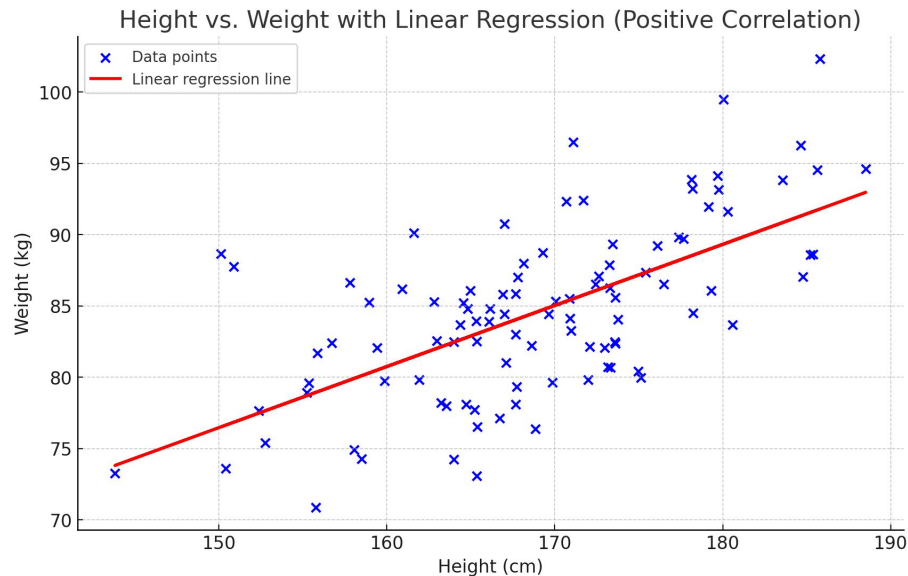


Linear regression example

- Heights vs weights
- It has positive correlation

- Function Class (Model)

$$\mathcal{G} = \{g_{a,b}(x) = ax + b\}$$
$$\theta = (a, b)$$

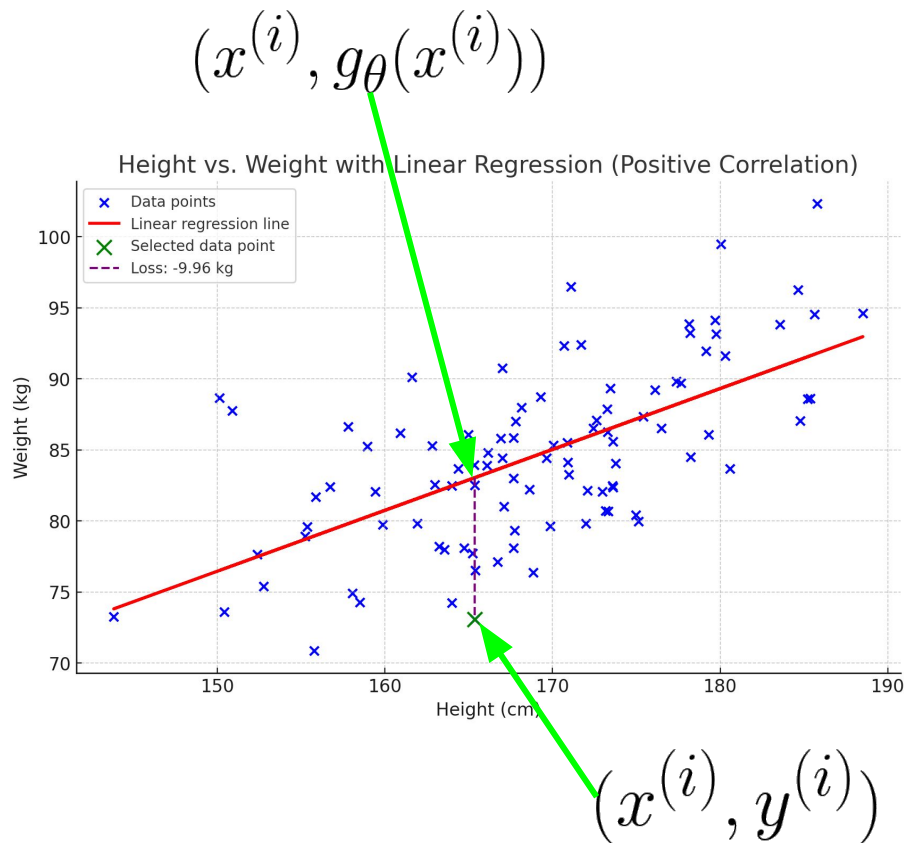


Linear regression example

- Loss function

$$\begin{aligned} & \ell(g_{\theta}(x^{(i)}), y^{(i)}) \\ &= (g_{\theta}(x^{(i)}) - y^{(i)})^2 \\ &= ((ax^{(i)} + b) - y^{(i)})^2 \end{aligned}$$

Mean-Squared Error (MSE)



Linear regression example

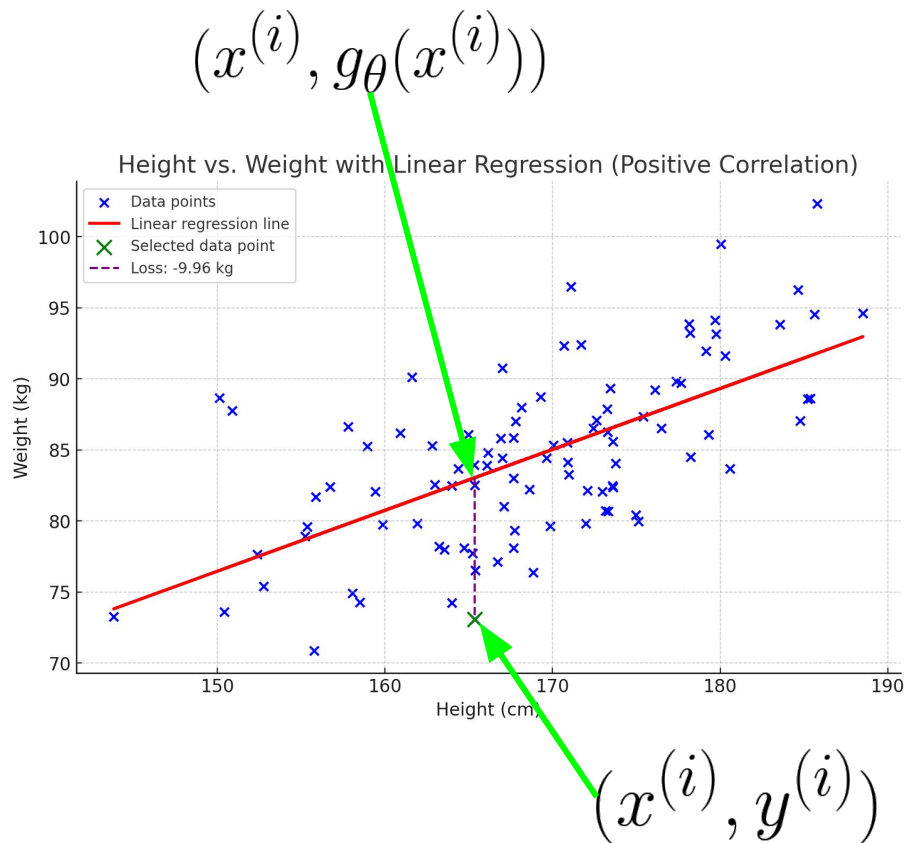
- Why not absolute difference?

$$\ell(g_{\theta}(x^{(i)}), y^{(i)}) = |g_{\theta}(x^{(i)}) - y^{(i)}|$$

- Why not perpendicular distance?

It is perfectly fine, but

- MSE is differentiable and has analytic solution



Solve Linear Regression

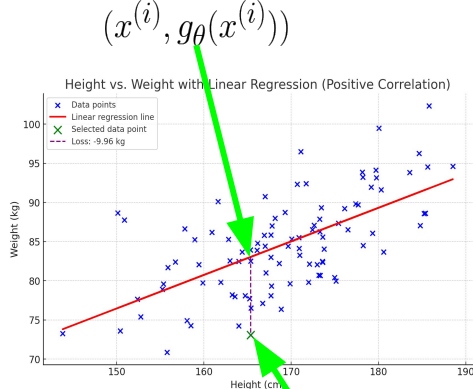
- Loss function

$$L(a, b) = \frac{1}{3} \left[(y^{(1)} - (ax^{(1)} + b))^2 + (y^{(2)} - (ax^{(2)} + b))^2 + (y^{(3)} - (ax^{(3)} + b))^2 \right]$$

- Simplified

$$L(a, b) = \frac{1}{3} \left[\sum_{i=1}^3 (y^{(i)})^2 - 2a \sum_{i=1}^3 y^{(i)} x^{(i)} - 2b \sum_{i=1}^3 y^{(i)} + a^2 \sum_{i=1}^3 (x^{(i)})^2 + 2ab \sum_{i=1}^3 x^{(i)} + 3b^2 \right]$$

$$L(a, b) = \frac{1}{3} [C - 2aS_{xy} - 2bS_y + a^2S_{xx} + 2abS_x + 3b^2]$$



Solve Linear Regression

$$\begin{aligned}\frac{\partial L}{\partial a} &= \frac{1}{3} (-2S_{xy} + 2aS_{xx} + 2bS_x) \\ &= \frac{2}{3} (aS_{xx} + bS_x - S_{xy})\end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial b} &= \frac{1}{3} (-2S_y + 2aS_x + 6b) \\ &= \frac{2}{3} (aS_x + 3b - S_y)\end{aligned}$$

$$\nabla L(a, b) = \left(\frac{\partial L}{\partial a}, \frac{\partial L}{\partial b} \right)$$

$$\nabla L(a, b) = \left(\frac{2}{3} (aS_{xx} + bS_x - S_{xy}), \frac{2}{3} (aS_x + 3b - S_y) \right)$$

Summary: Supervised Learning and Linear Regression

- dataset (with label)
- Set function class $\mathcal{G}$ and loss function ℓ
- Minimizes

$$\mathcal{L}(\theta) = \sum_{i=1}^n \ell(g_{\theta}(x^{(i)}), y^{(i)})$$

- Linear Regression

$$\mathcal{G} = \{g_{a,b}(x) = ax + b\}$$

$$\nabla L(a, b) = \left(\frac{\partial L}{\partial a}, \frac{\partial L}{\partial b} \right)$$

감사합니다

지도학습 2강

— Linear Regression —

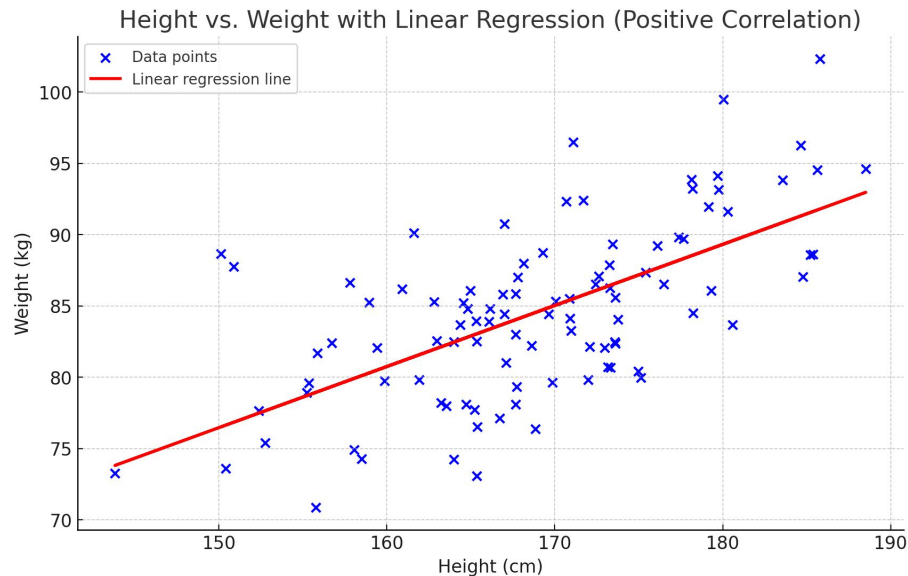
Linear regression (revisited)

- Heights vs weights
- Function Class (Model)

$$\mathcal{G} = \{g_{a,b}(x) = ax + b\}$$
$$\theta = (a, b)$$

- Loss Function

$$\begin{aligned} & \ell(g_{\theta}(x^{(i)}), y^{(i)}) \\ &= (g_{\theta}(x^{(i)}) - y^{(i)})^2 \\ &= ((ax^{(i)} + b) - y^{(i)})^2 \end{aligned}$$



Linear regression (Multidimension)

- (heights, hand size) vs. weights

$$x = (x_1, x_2) \quad x \in \mathcal{X} = \mathbb{R}^2, y \in \mathcal{Y} = \mathbb{R}$$

- Model Class

$$\mathcal{G} = \{g_{a_1, a_2, b}(x) = a_1 x_1 + a_2 x_2 + b\}$$

$$\theta = (a_1, a_2, b)$$

- Loss function
- $$\begin{aligned} & \ell(g_\theta(x^{(i)}), y^{(i)}) \\ &= (g_\theta(x^{(i)}) - y^{(i)}) \\ &= ((a_1 x_1^{(i)} + a_2 x_2^{(i)} + b) - y^{(i)}) \end{aligned}$$

Linear regression (Multidimension)

$$L(a_1, a_2, b) = \frac{1}{3} \sum_{i=1}^3 \left(y^{(i)} - (a_1 x_1^{(i)} + a_2 x_2^{(i)} + b) \right)^2$$

$$L(a_1, a_2, b) = \frac{1}{3} \sum_{i=1}^3 \left[(y^{(i)})^2 - 2y^{(i)}a_1x_1^{(i)} - 2y^{(i)}a_2x_2^{(i)} - 2y^{(i)}b + (a_1x_1^{(i)})^2 + 2a_1a_2x_1^{(i)}x_2^{(i)} + 2a_1x_1^{(i)}b + (a_2x_2^{(i)})^2 + 2a_2x_2^{(i)}b + b^2 \right]$$

- Again a quadratic function

$$L(a_1, a_2, b) = \frac{1}{3} [C - 2a_1S_{x_1y} - 2a_2S_{x_2y} - 2bS_y + a_1^2S_{x_1x_1} + 2a_1a_2S_{x_1x_2} + 2a_1bS_{x_1} + a_2^2S_{x_2x_2} + 2a_2bS_{x_2} + 3b^2]$$

- Gradient = 0

$$\nabla L(a_1, a_2, b) = \left(\frac{\partial L}{\partial a_1}, \frac{\partial L}{\partial a_2}, \frac{\partial L}{\partial b} \right)$$

Linear regression (Multidimension)

Partial Derivative with Respect to a_1

$$\frac{\partial L}{\partial a_1} = \frac{2}{3} (-S_{x_1 y} + a_1 S_{x_1 x_1} + a_2 S_{x_1 x_2} + b S_{x_1})$$

Partial Derivative with Respect to a_2

$$\frac{\partial L}{\partial a_2} = \frac{2}{3} (-S_{x_2 y} + a_2 S_{x_2 x_2} + a_1 S_{x_1 x_2} + b S_{x_2})$$

Partial Derivative with Respect to b

$$\frac{\partial L}{\partial b} = \frac{2}{3} (-S_y + a_1 S_{x_1} + a_2 S_{x_2} + 3b)$$

Linear regression (Multidimension)

- data

$$x = (x_1, x_2, \dots, x_d) \in \mathbb{R}^d, y \in \mathcal{Y} = \mathbb{R}$$

- Function Class

$$\mathcal{G} = \{g_{a,b}(x) = a^\top x + b : a \in \mathbb{R}^d, b \in \mathbb{R}\}$$
$$\theta = (a_1, a_2, \dots, a_d, b)$$

- Loss function

$$\ell(g_{a,b}(x^{(i)}), y^{(i)}) = (g_{a,b}(x^{(i)}) - y^{(i)})^2$$

Linear regression (Multidimension)

- data $x_0 = 1$

$$x = (1, x_1, x_2, \dots, x_d) \in \mathbb{R}^{d+1}, y \in \mathbb{R}$$

- Function Class

$$\mathcal{G} = \{g_a = a^\top x : a \in \mathbb{R}^{d+1}\}$$

$$\theta = (a_0, a_1, a_2, \dots, a_d)$$

- Loss function

$$\ell(g_a(x^{(i)}), y^{(i)}) = (g_a(x^{(i)}) - y^{(i)})^2$$

$$\mathcal{L}(a) = \frac{1}{n} \sum_{i=1}^n (g_a(x^{(i)}) - y^{(i)})^2$$

Linear regression (Multidimension)

- Minimize

$$\mathcal{L}(a) = \frac{1}{n} \sum_{i=1}^n (g_a(x^{(i)}) - y^{(i)})^2$$

- Gradient

$$\nabla \mathcal{L}(a) = \frac{1}{n} \sum_{i=1}^n \nabla (g_a(x^{(i)}) - y^{(i)})^2$$

- Can solve linear equations with $n+1$ variables

Normal Equation

$$X = \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(n)} \end{bmatrix} \in \mathbb{R}^{n \times d}, Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix} \in \mathbb{R}^n$$

$$\begin{aligned} \mathcal{L}(a) &= \left\| \begin{bmatrix} a^\top x^{(1)} - y^{(1)} \\ a^\top x^{(2)} - y^{(2)} \\ \vdots \\ a^\top x^{(n)} - y^{(n)} \end{bmatrix} \right\|^2 = \|Xa - Y\|^2 \\ &= Y^\top Y - 2Y^\top X\mathbf{a} + \mathbf{a}^\top X^\top X\mathbf{a} \end{aligned}$$

Normal Equation

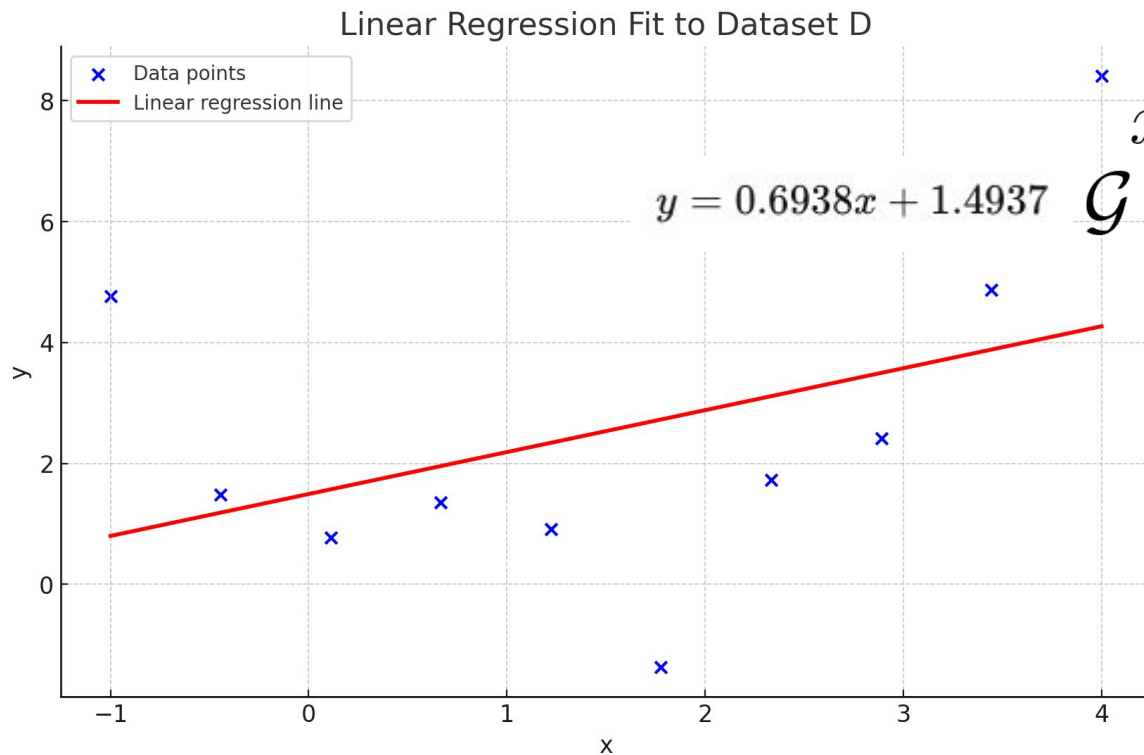
$$\mathcal{L}(a) = \left\| \begin{bmatrix} a^\top x^{(1)} - y^{(1)} \\ a^\top x^{(2)} - y^{(2)} \\ \vdots \\ a^\top x^{(n)} - y^{(n)} \end{bmatrix} \right\|^2 = \|Xa - Y\|^2$$
$$= Y^T Y - 2Y^T X \mathbf{a} + \mathbf{a}^T X^T X \mathbf{a}$$

$$\frac{\partial \mathcal{L}(\mathbf{a})}{\partial \mathbf{a}} = -2X^T Y + 2X^T X \mathbf{a} = 0$$

$$X^T X \mathbf{a} = X^T Y$$

$$\mathbf{a} = (X^T X)^{-1} X^T Y$$

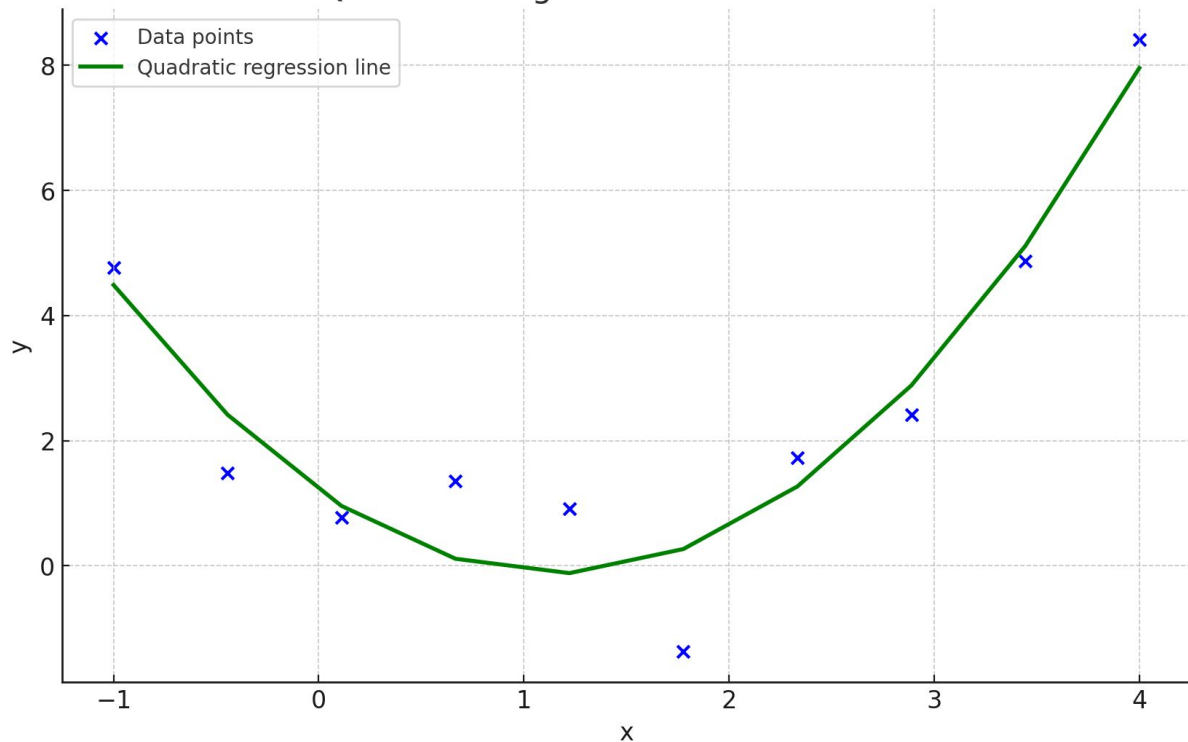
What if we want to fit quadratic equation?



$$x \in \mathcal{X} = \mathbb{R}, y \in \mathcal{Y} = \mathbb{R}$$
$$\mathcal{G} = \{g_{a,b}(x) = ax + b\}$$

What if we want to fit quadratic equation?

Quadratic Regression Fit to Dataset D



$$y = 1.1986 - 2.2945x + 0.9961x^2$$

Is this quadratic regression?
→ No

We fit parameter “a”

- Redefine the dataset

$$\tilde{x} = (1, x, x^2) \in \mathbb{R}^3, y \in \mathbb{R}$$

- Model Class

$$\mathcal{G} = \{g_a(x) = a^\top \tilde{x} : a \in \mathbb{R}^3\}$$

$$g_a(x) = a_0 + a_1x + a_2x^2$$

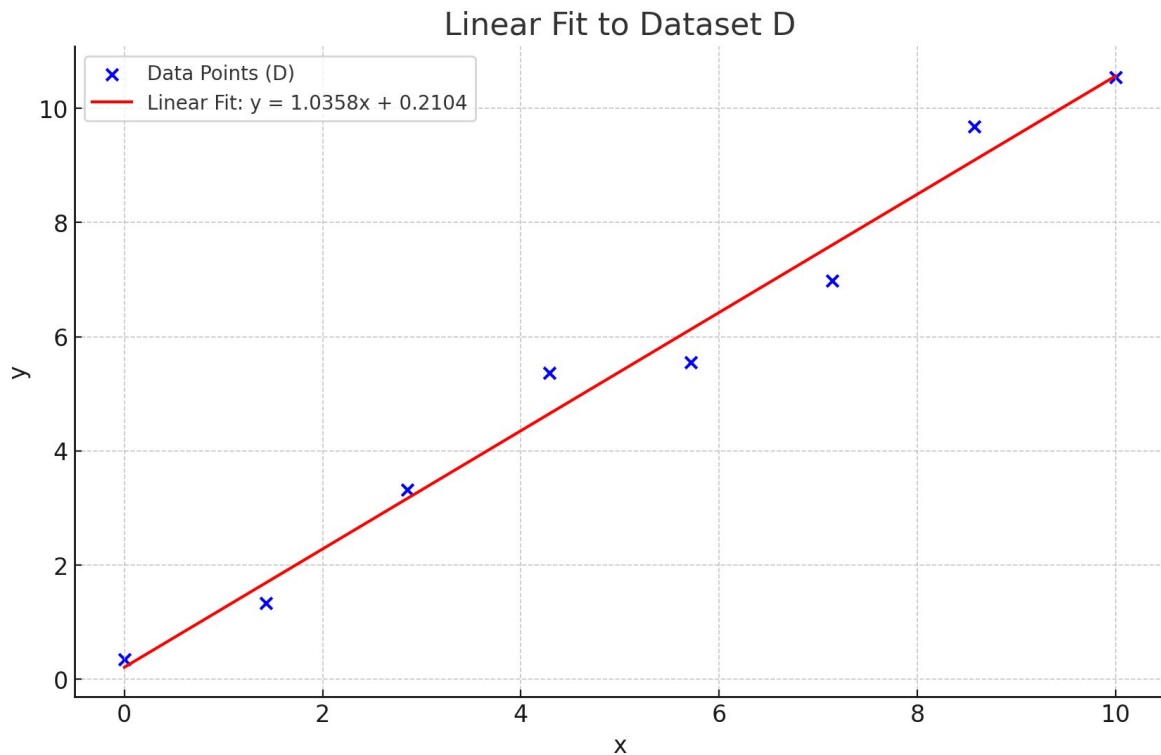
- Loss function: the same MSE

$$\mathcal{L}(a) = \sum_{i=1}^n ((a_0 + a_1x^{(i)} + a_2(x^{(i)})^2) - y^{(i)})^2$$

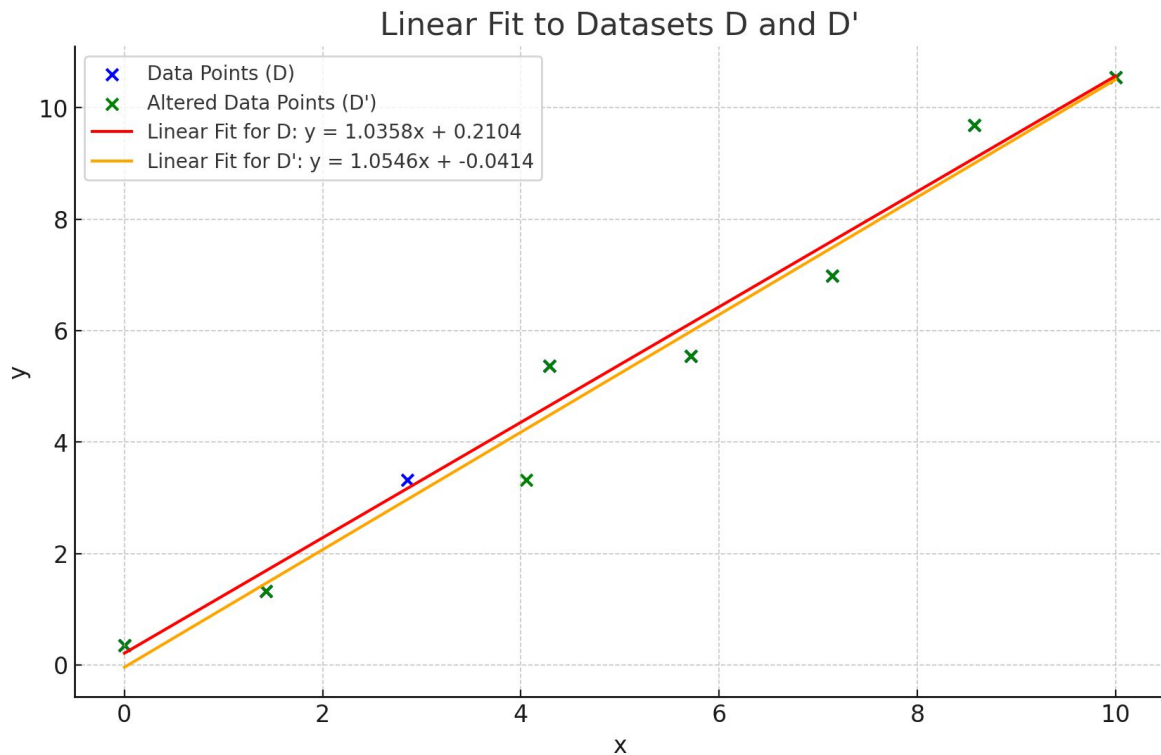
You can always add features

- For multidimensional input $x \in \mathbb{R}^d$
We can add $\tilde{x} = (x_1, \dots, x_d, k_1(x), k_2(x), \dots)$
- More feature, more expressive
But should be careful (**overfitting!**)
- Features can be selected via inspection or based on expertises

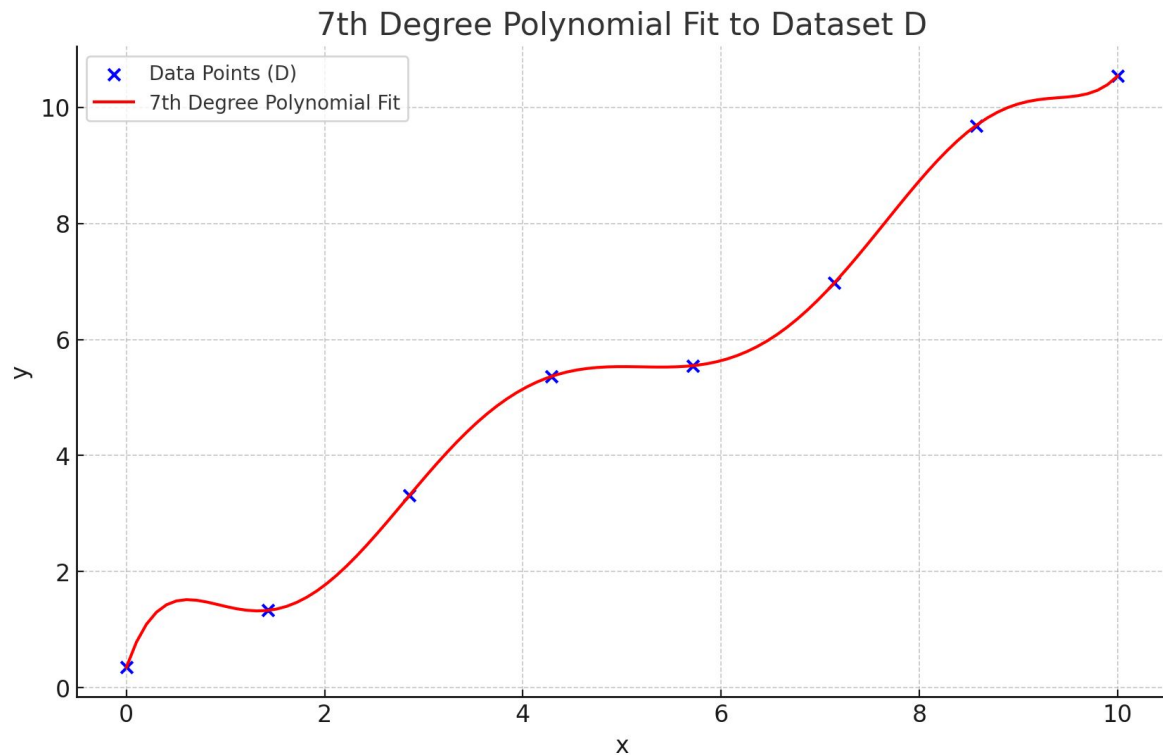
Overfitting: Bias-Variance Tradeoff



Overfitting: Bias-Variance Tradeoff

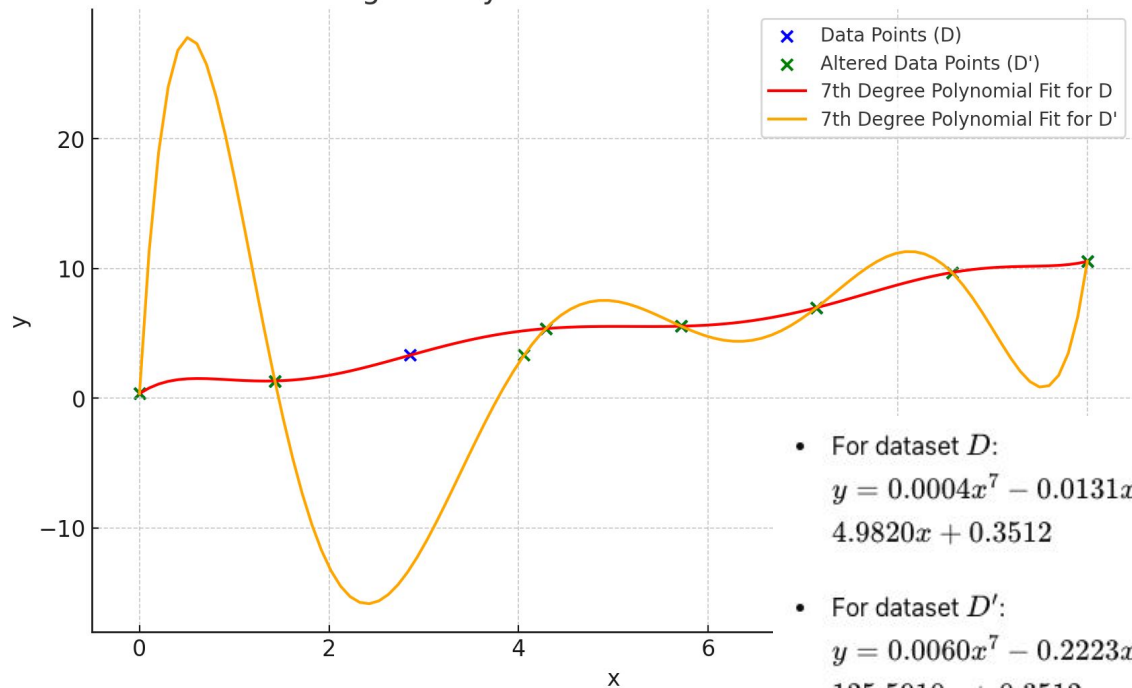


Overfitting: Bias-Variance Tradeoff



Overfitting: Bias-Variance Tradeoff

7th Degree Polynomial Fit to Datasets D and D'



- For dataset D :

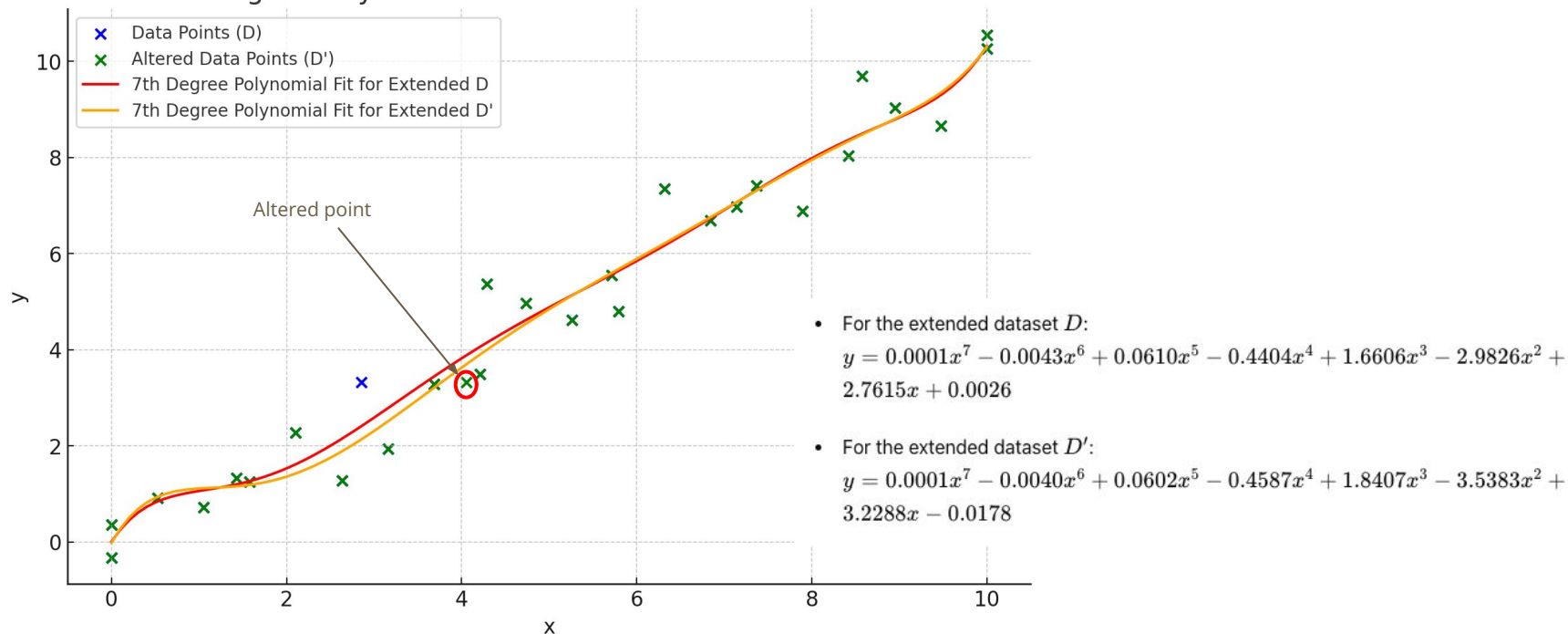
$$y = 0.0004x^7 - 0.0131x^6 + 0.1867x^5 - 1.3032x^4 + 4.5845x^3 - 7.3889x^2 + 4.9820x + 0.3512$$

- For dataset D' :

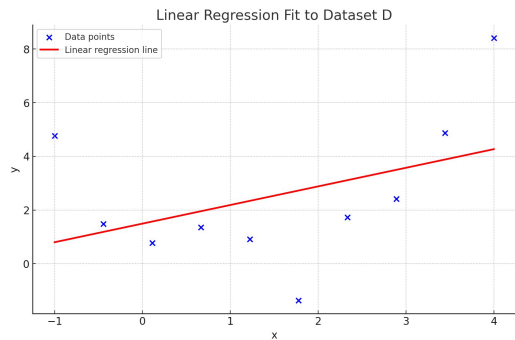
$$y = 0.0060x^7 - 0.2223x^6 + 3.2894x^5 - 24.6142x^4 + 96.7265x^3 - 184.0811x^2 + 125.5910x + 0.3512$$

More data is better

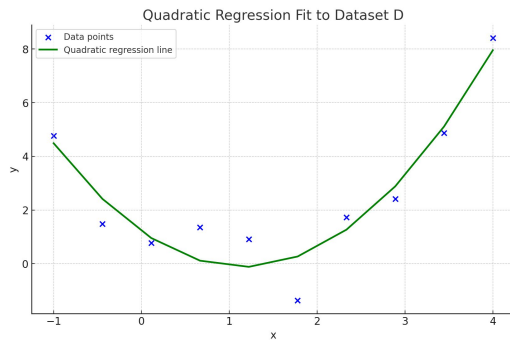
7th Degree Polynomial Fit to Extended Datasets D and D'



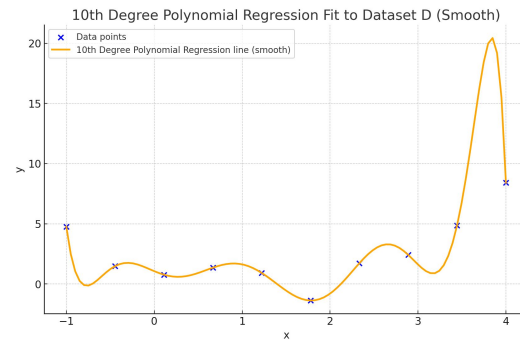
Underfitting vs. Overfitting



Underfit
High bias



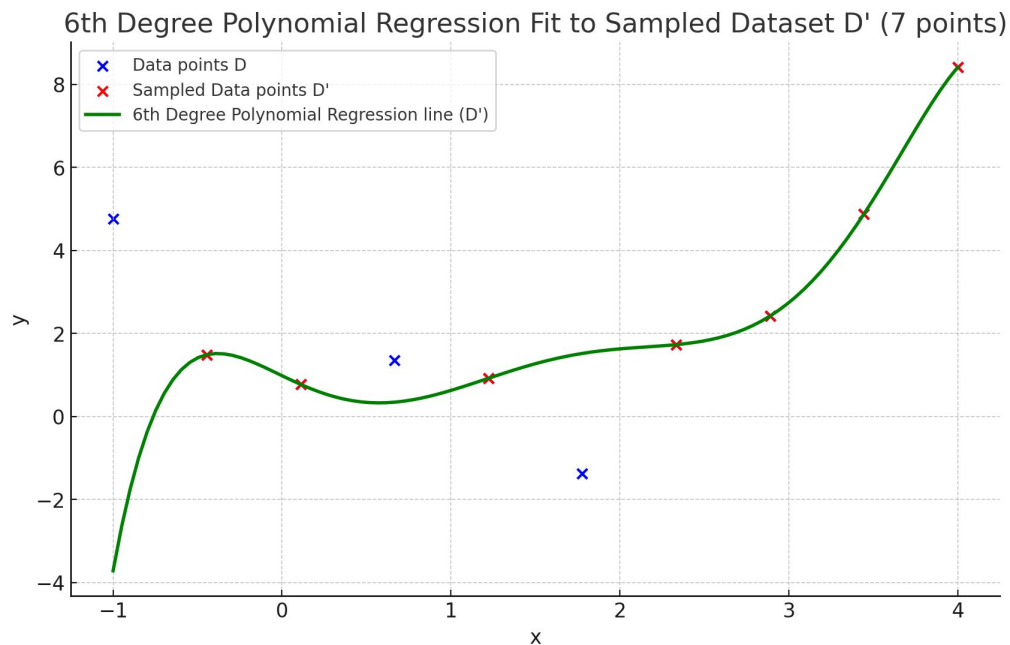
Just right



Overfit
High variance

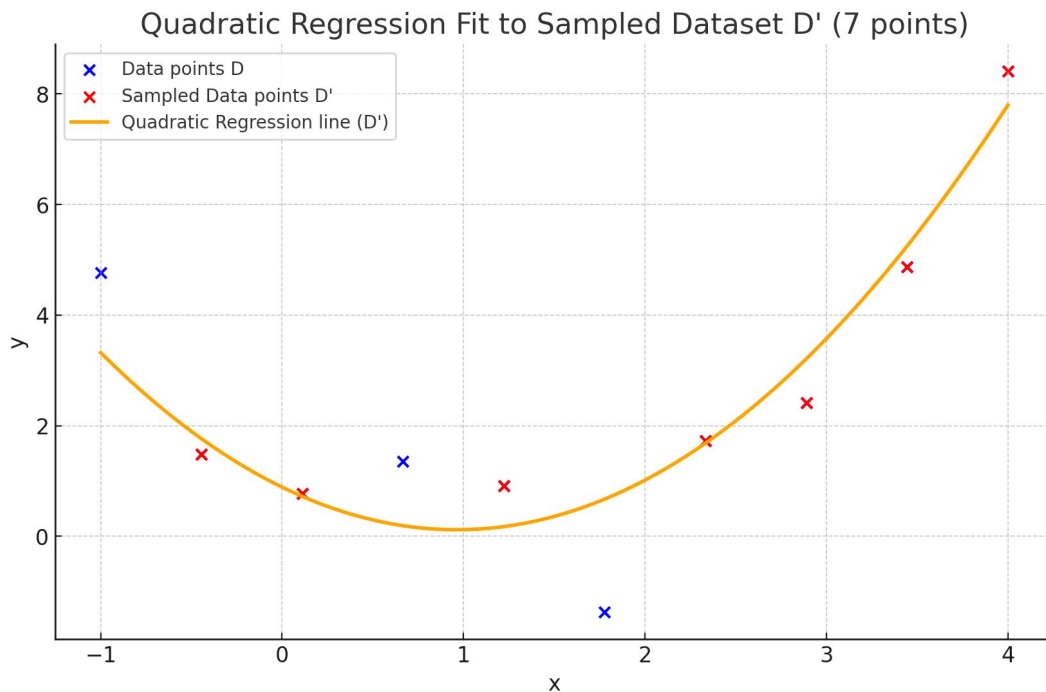
How can we recognize overfitting?

- Data is often high dimensional
 - Hard to visualize
- Use validation set
 - train set (red)
 - validation set (blue)
- Train loss ~ 0
- Validation loss $>$ Train loss



Train Loss ~ Validation Loss

- Generalization
 - Train loss ~ Validation loss



Train-Validation-Test

- Train set: roughly 80% of data
 - Fit your model
 - Validation set: remaining 20% of data
 - Validate your model
 - Test set: another separate dataset
 - Test your model
-
- Test set is often unknown in training stage (think about competition)
 - Validation also influences the decision

Rules

- Based on prior knowledge (or theory)
 - Occam's Razor

The explanation that requires the fewest assumptions is usually correct.
 - Inspect data
- Low training error
- Train error ~ Validation error: good
- Train error $\ll$ Validation error: overfit

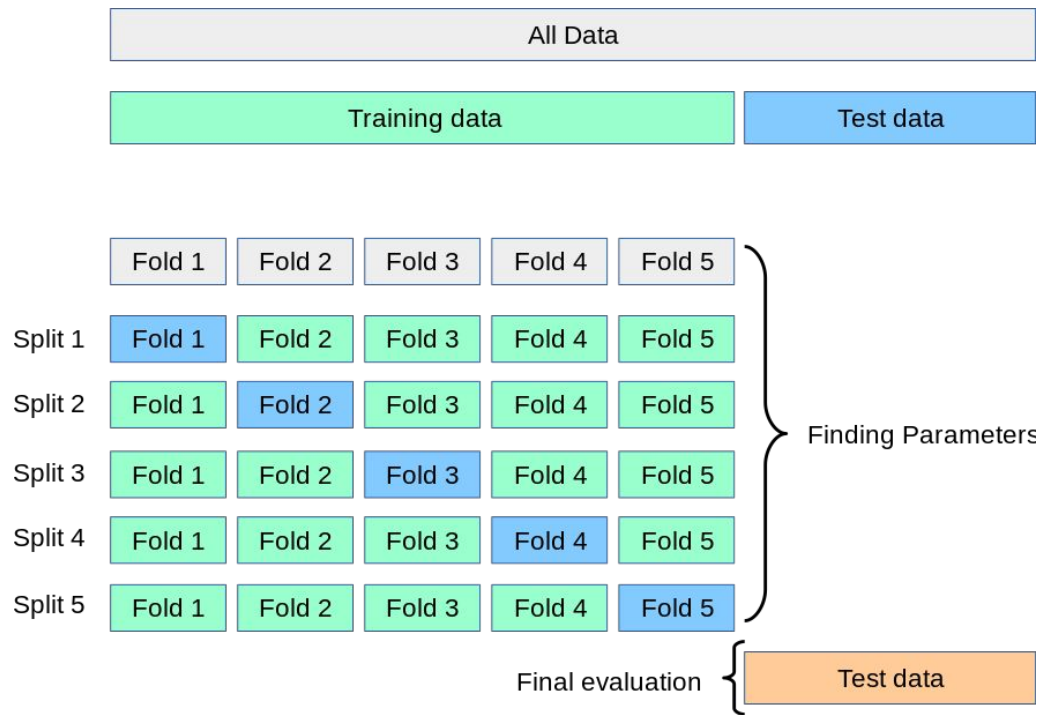


If you are in overfit

- Obtain more data
 - Expensive
- Generate samples with generative models (careful)
- Regularization
 - L2 term
- Augmentation
 - Add more data

Cross Validation

- If you lack data



Regularization

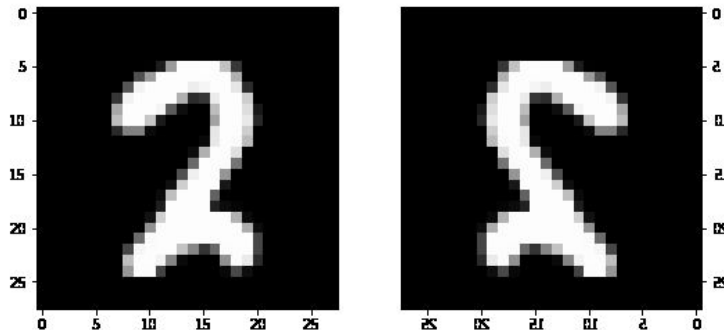
- Add regularization term (for example, L2)

$$\mathcal{L}(a) + \lambda \mathcal{L}_{reg}(a) = \sum_{i=1}^n \ell(g_{\theta}(x^{(i)}), y^{(i)}) + \lambda \|a\|^2$$

- It prevents overfitting
 - Large λ : regularize more
 - Small λ : regularize less

Augmentation

- Shifted dog is also a dog
 - Translation invariant
- You can also use
 - Flip
 - Enlarge
 - Rotate
 - Add noise
 - Color jitter, etc
- But it depends on your dataset
 - Horizontal flip of '2' is not '2'.



Summary: Linear Regression

- Normal Equation

$$X^T X \mathbf{a} = X^T Y$$

$$\mathbf{a} = (X^T X)^{-1} X^T Y$$

$$\mathcal{L}(a) = \left\| \begin{bmatrix} a^\top x^{(1)} - y^{(1)} \\ a^\top x^{(2)} - y^{(2)} \\ \vdots \\ a^\top x^{(n)} - y^{(n)} \end{bmatrix} \right\|^2 = \|Xa - Y\|^2$$

- Overfitting vs Underfitting
- Train vs Validation
- Augmentation

감사합니다

지도학습 3강

Gradient Descent

Supervised Learning Revisited

- Given dataset (with label)

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots (x^{(n)}, y^{(n)})$$

- Set function class $\mathcal{G}$
- Set loss function ℓ
- Find $g_{\theta} \in \mathcal{G}$ that minimizes

$$\mathcal{L}(\theta) = \sum_{i=1}^n \ell(g_{\theta}(x^{(i)}), y^{(i)})$$

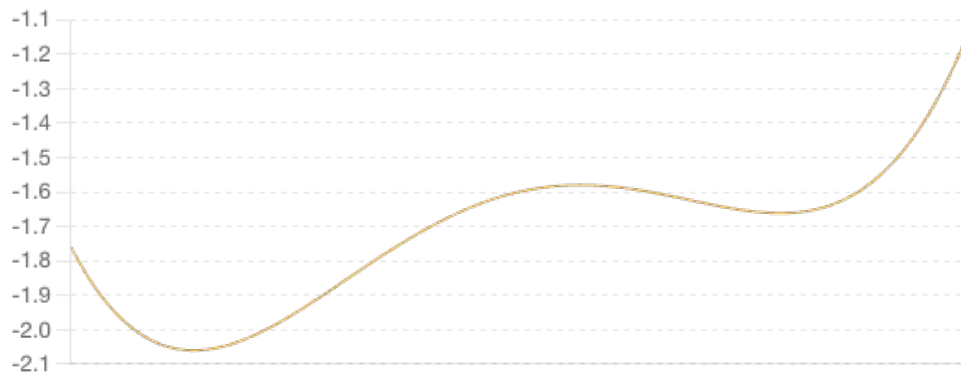
Supervised Learning Revisited

- Function class can be complicated
 - Deep Neural Networks
 - Parameter can easily be millions, even billions
- Loss function can be complex
 - Analytic solution may not be available
- How can we optimize then?

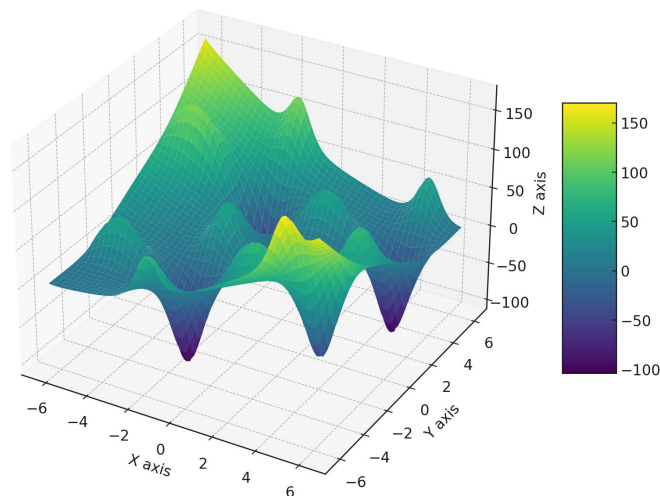
$$\mathcal{L}(\theta) = \sum_{i=1}^n \ell(g_{\theta}(x^{(i)}), y^{(i)})$$

How can we find minimum

- High school math:
 - Minimizing $f \rightarrow \text{find } f'(x) = 0$
- Minimizing loss function,
 - Find $\nabla \mathcal{L}(\theta) = 0$
- Note that
 - Such theta can be local minimum



3D Surface Plot of $z = \sin(x) \cdot \exp((1 - \cos(y))^2) + \cos(y) \cdot \exp((1 - \sin(x))^2) + (x - y)^2$



Gradient Descent

- Assumption
 - We know gradient
 - But does not have global view
- Best you can do is take the slope!

Gradient Descent

- Assumption
 - We know gradient
 - But does not have global view

- Algorithm

- Initialize at random point θ_0
- At i-th step,
 - compute gradient $\nabla \mathcal{L}(\theta_i)$
 - Update parameter

$$\theta_{i+1} = \theta_i - \alpha \nabla \mathcal{L}(\theta_i)$$

Steepest increasing direction

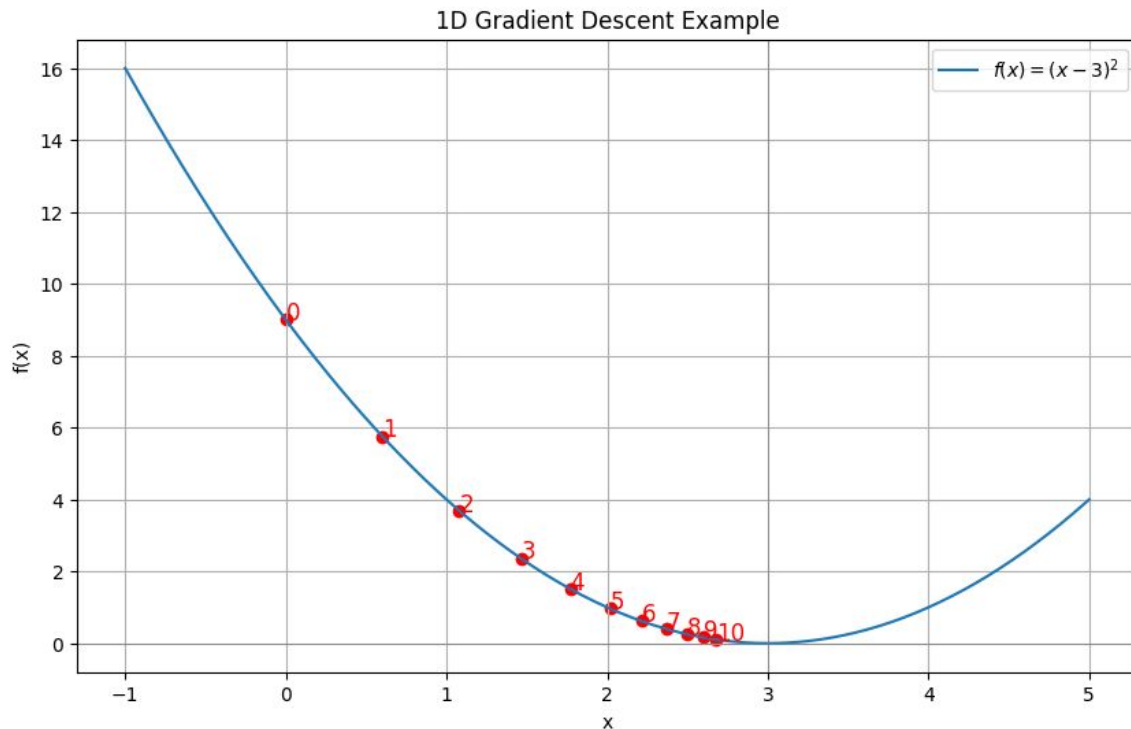


Learning rate



Assumption

- $f(x) = (x - 3)^2$
- $x_0 = 0$
- $f'(x) = 2(x - 3)$
- $\alpha = 0.1$



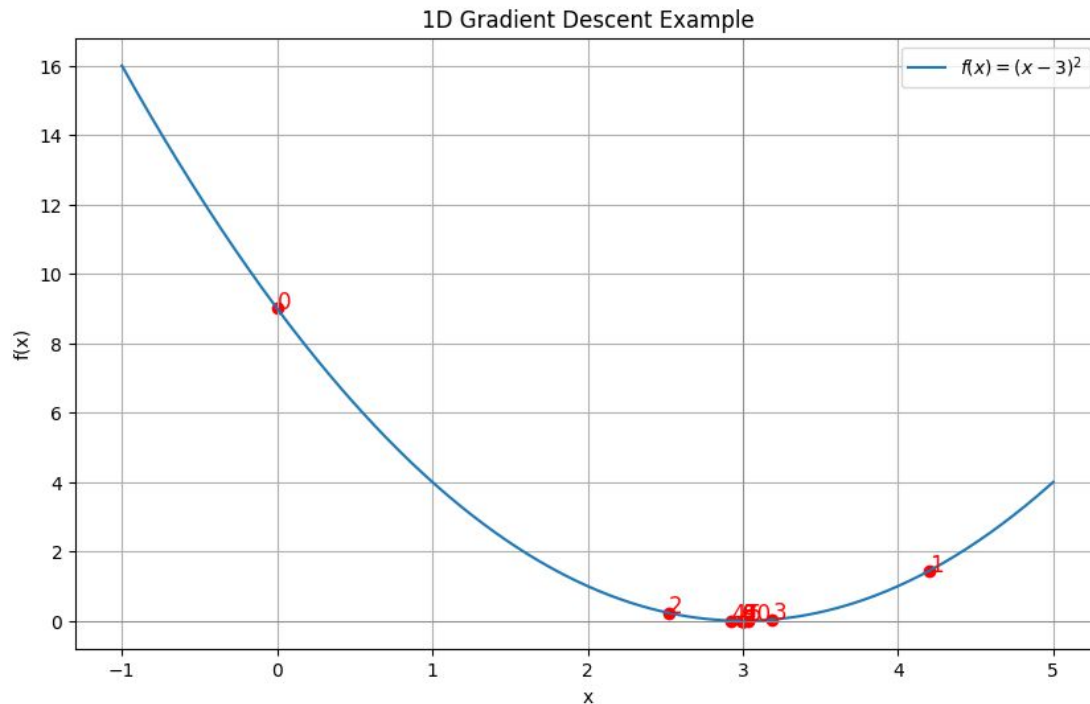
Learning rate

- $f(x) = (x - 3)^2$

- $x_0 = 0$

$$f'(x) = 2(x - 3)$$

$$\alpha = 0.7$$



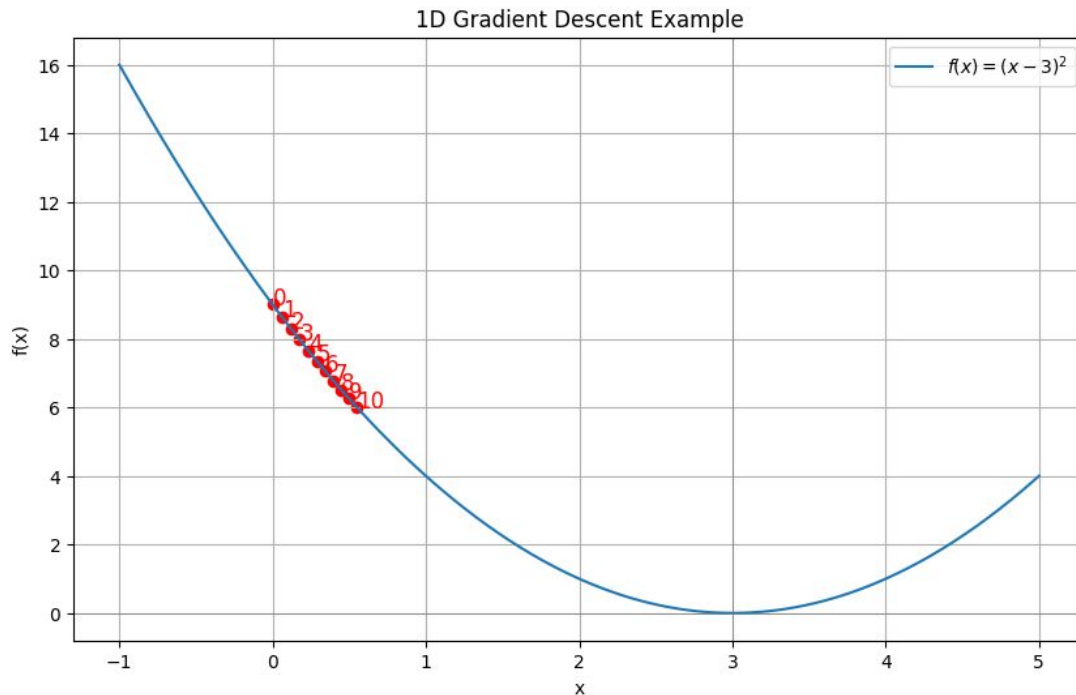
Small learning rate

- $f(x) = (x - 3)^2$

- $x_0 = 0$

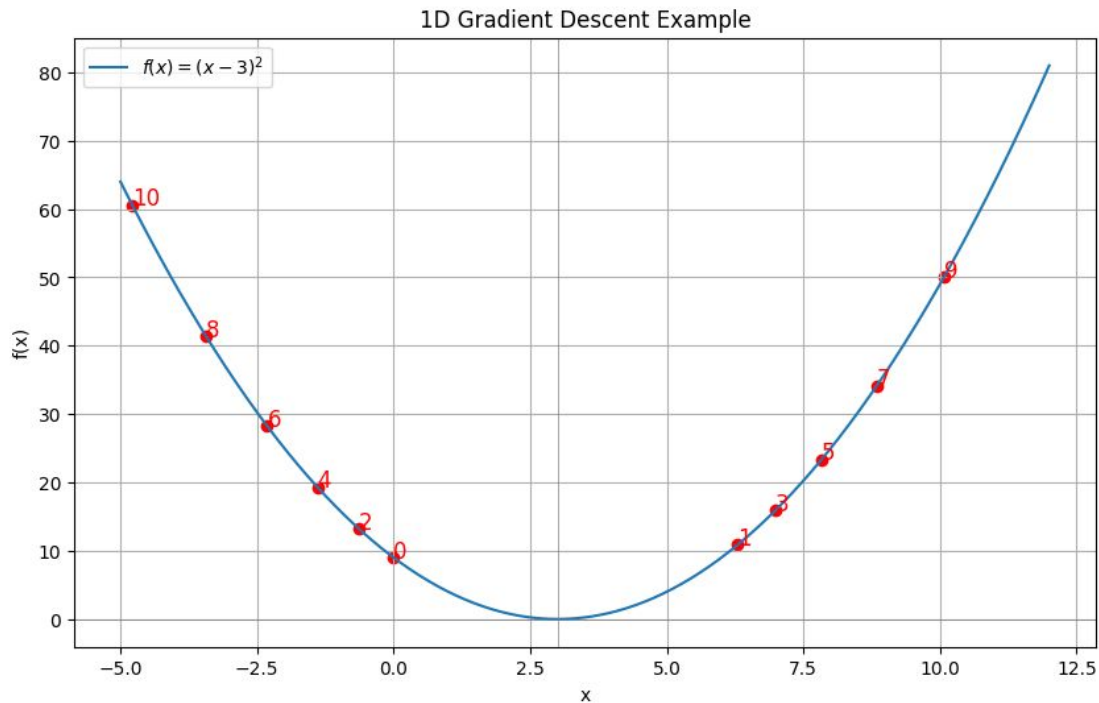
$$f'(x) = 2(x - 3)$$

$$\alpha = 0.01$$



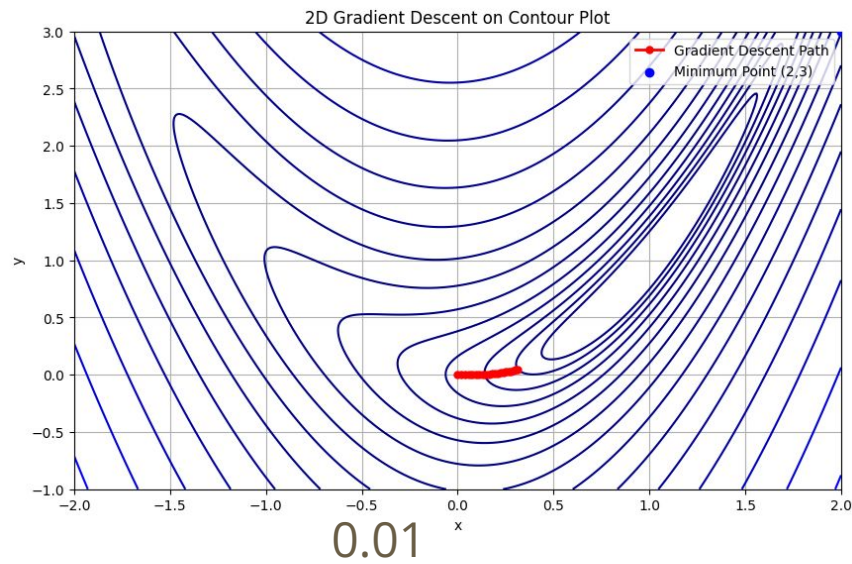
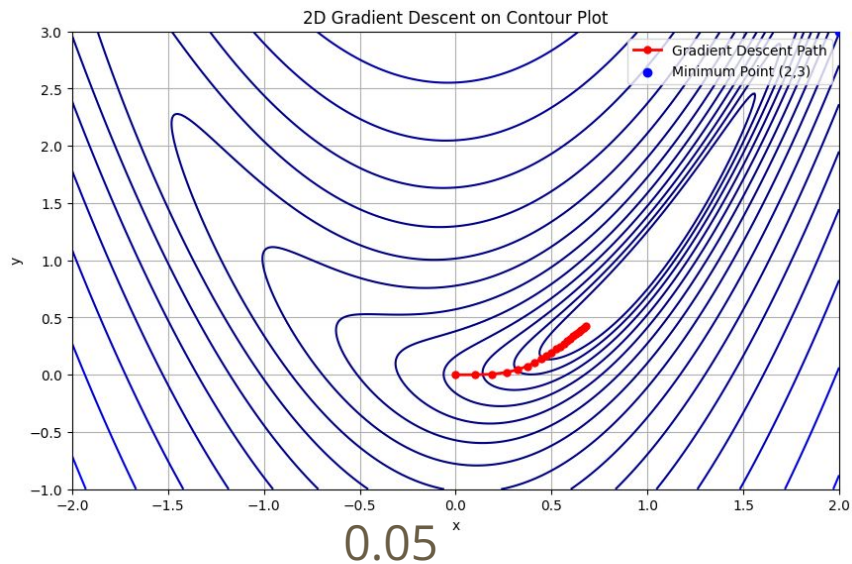
Large learning rate

- $f(x) = (x - 3)^2$
- $x_0 = 0$
- $f'(x) = 2(x - 3)$
- $\alpha = 1.05$



2D Gradient Descent

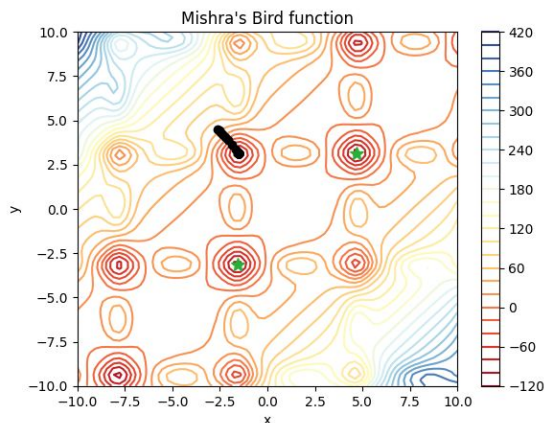
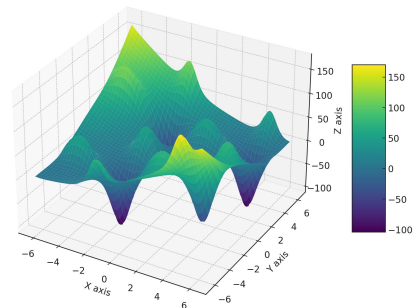
- $f(x, y) = (1 - x)^2 + 5(y - x^2)^2$
- $\nabla f(x, y) = (2(x - 1) + 20(x^2 - y)x, 10(y - x^2))$



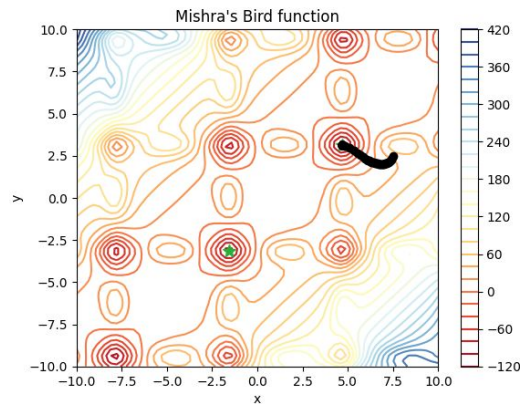
2D Gradient Descent

- $f(x, y) = \sin(x) \exp((1 - \cos(y))^2) + \cos(y) \exp((1 - \sin(x))^2) + (x - y)^2$
- Gradient:

$$\begin{pmatrix} \cos(x) \exp((1 - \cos(y))^2) + \cos(y) \exp((1 - \sin(x))^2) \cdot 2(1 - \sin(x))(-\cos(x)) + 2(x - y) \\ \sin(x) \exp((1 - \cos(y))^2) \cdot 2(1 - \cos(y)) \sin(y) - \sin(y) \exp((1 - \sin(x))^2) + 2(y - x) \end{pmatrix}$$



Initialization 1



Initialization 2

Gradient Descent

- Pick several random initialization
- Adjust learning rate
 - If loss decays too slow: increase lr
 - If loss diverge: decrease lr

Gradient Descent in Practice

- There are automatic tools for gradients
 - You do not need to compute
- However, you may have too many data points

$$\mathcal{L}(\theta) = \sum_{i=1}^n \ell(g_{\theta}(x^{(i)}), y^{(i)})$$

- In this case, we sample and compute the gradient
 - Gradient is linear!
 - Stochastic Gradient Descent

Gradient Descent

Algorithm 1 Gradient Descent

- 1: Initialize parameters (weights) θ randomly
 - 2: Set learning rate α
 - 3: **repeat**
 - 4: Compute the gradient of the cost function with respect to θ :
 - 5: $\nabla \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla \ell(g_{\theta}(x^{(i)}), y_i)$
 - 6: Update the parameters: $\theta := \theta - \alpha \nabla \mathcal{L}(\theta)$
 - 7: **until** convergence
-

Stochastic Gradient Descent

Algorithm 2 Stochastic Gradient Descent

- 1: Initialize parameters (weights) θ randomly
 - 2: Set learning rate α
 - 3: **repeat**
 - 4: **for** each training example (x_i, y_i) **do**
 - 5: Compute the gradient of the cost function with respect to θ for the current example:
6: $\nabla \mathcal{L}(\theta; x^{(i)}, y^{(i)}) = \nabla \ell(g_\theta(x^{(i)}), y_i)$
 - 7: Update the parameters: $\theta := \theta - \alpha \nabla \mathcal{L}(\theta; x^{(i)}, y^{(i)})$
 - 8: **end for**
 - 9: **until** convergence
-

Mini-batch Gradient Descent

Algorithm 3 Mini-batch Gradient Descent

- 1: Initialize parameters (weights) θ randomly
 - 2: Set learning rate α , mini-batch size b
 - 3: **repeat**
 - 4: **for** each mini-batch B of size b **do**
 - 5: Compute the gradient of the cost function with respect to θ for the mini-batch:
6: $\nabla \mathcal{L}(\theta; B) = \frac{1}{b} \sum_{i \in B} \nabla \ell(g_{\theta}(x^{(i)}), y_i)$
 - 7: Update the parameters: $\theta := \theta - \alpha \nabla \mathcal{L}(\theta; B)$
 - 8: **end for**
 - 9: **until** convergence
-

Gradient Descent - Linear Regression

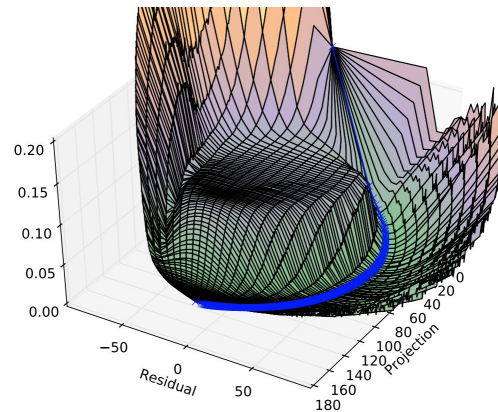
$$\begin{aligned}\nabla \ell(g_{\theta}(x^{(i)}), y^{(i)}) &= \nabla (\theta^{\top} x^{(i)} - y^{(i)})^2 \\ &= (\theta^{\top} x^{(i)} - y^{(i)}) x^{(i)}\end{aligned}$$

Algorithm 4 Gradient Descent (Linear Regression)

- 1: Initialize parameters (weights) θ randomly
 - 2: Set learning rate α
 - 3: **repeat**
 - 4: Compute the gradient of the cost function with respect to θ :
 - 5: $\nabla \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (g_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$
 - 6: Update the parameters: $\theta := \theta - \alpha \nabla \mathcal{L}(\theta)$
 - 7: **until** convergence
-

Gradient Descent: Drawbacks

- Can stuck at plateau
- Can stuck at the local minima



QUALITATIVELY CHARACTERIZING NEURAL
NETWORK OPTIMIZATION PROBLEMS, ICLR 2015

Losslandscape.com

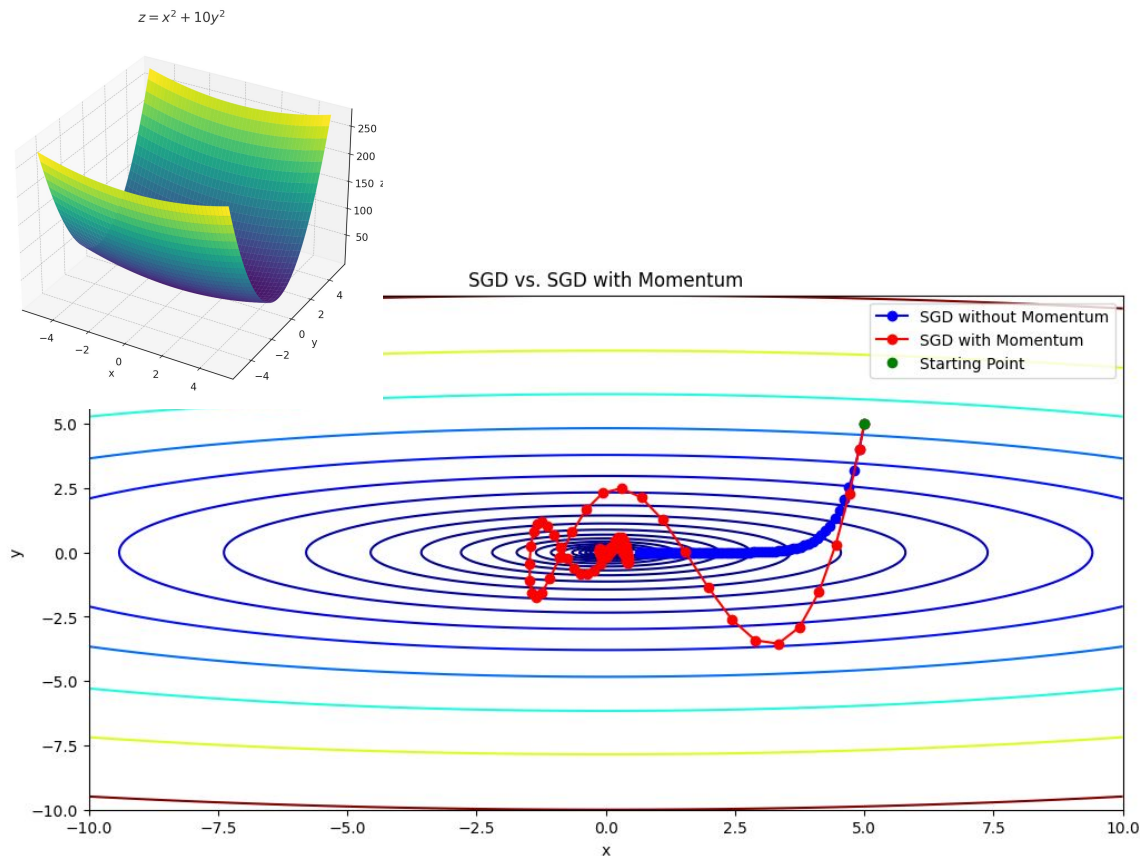
Momentum SGD

The cost function is:

$$J(\theta) = x^2 + 10y^2$$

The gradient of $J(\theta)$ is:

$$\nabla J(\theta) = \begin{bmatrix} 2x \\ 20y \end{bmatrix}$$



Momentum SGD

Weighted average of gradients

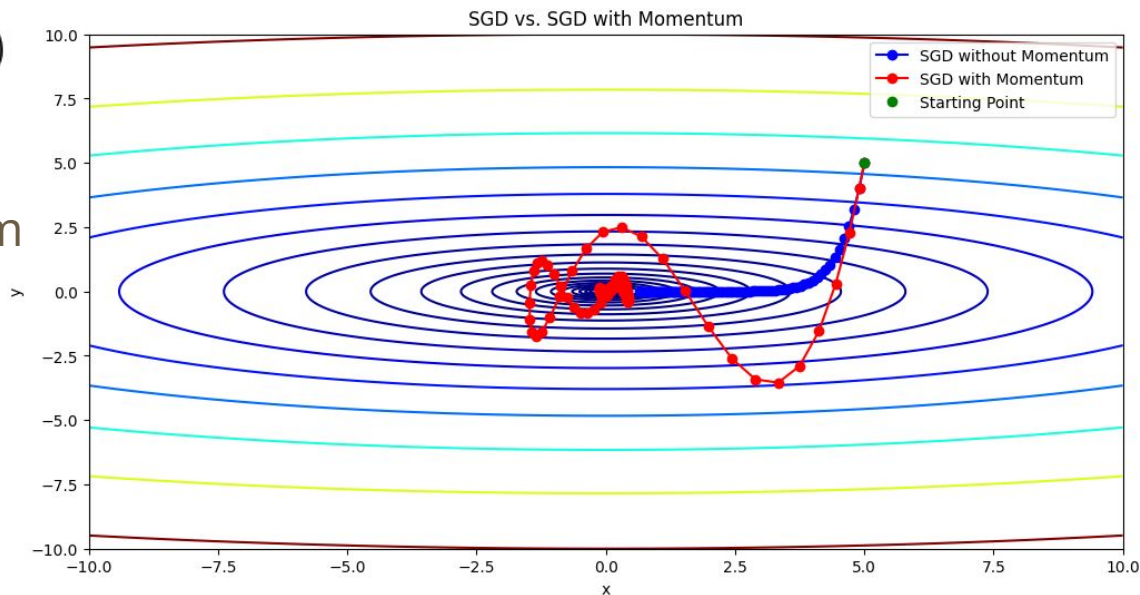
$$v_t = \beta v_{t-1} + (1 - \beta) \nabla \mathcal{L}(\theta_{t-1})$$

$\beta=0$ is regular GD

Larger β , higher momentum

Update Parameters

$$\theta_t = \theta_{t-1} - \alpha v_t$$



RMS Prop

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2$$

Element-wise square

$$\theta_t = \theta_{t-1} - \frac{\alpha}{\sqrt{E[g^2]_t + \epsilon}}g_t$$

Element-wise division

ADAM

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Bias correction

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias correction

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_t = \theta_{t-1} - \frac{\alpha \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Comparison

- Momentum SGD:
 - Uses a fixed LR
- RMSProp:
 - Adjusts the LR based on the average of recent gradients' magnitudes.
- ADAM:
 - Combines both momentum and adaptivity by using both the first moment (mean) and second moment (variance) of gradients.

Learning rate scheduling

Scheduling

Update learning rate every s steps:
if epoch % $s == 0$ **then**
 $\alpha := \alpha \times d$
end if

Exponential Scheduling

Update learning rate:

$$\alpha = \alpha_0 \times e^{-\gamma t}$$

Increment epoch:

$$t := t + 1$$

Adaptive Scheduling (Annealing)

Compute validation loss L_{val}
if $L_{\text{val}} < L_{\text{best}}$ **then**
 $L_{\text{best}} := L_{\text{val}}$
 Reset patience counter $k = 0$
else
 Increment patience counter $k := k + 1$
end if
if patience counter $k \geq p$ **then**
 Update learning rate:
 $\alpha := \alpha \times d$
 Reset patience counter $k = 0$
end if

Summary: Gradient Descent

- Take the slope
- Random initialization
- Learning Rate
- Momentum

감사합니다

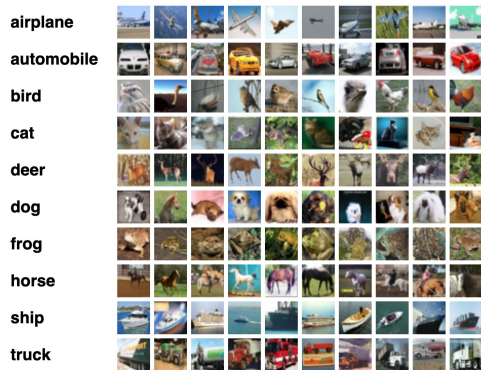
지도학습 4강

Classification

Classification vs. Regression

- Classification

Discrete (finite) $y \in \mathcal{Y}$



영어 - 감지됨

My dog is cute



Korean

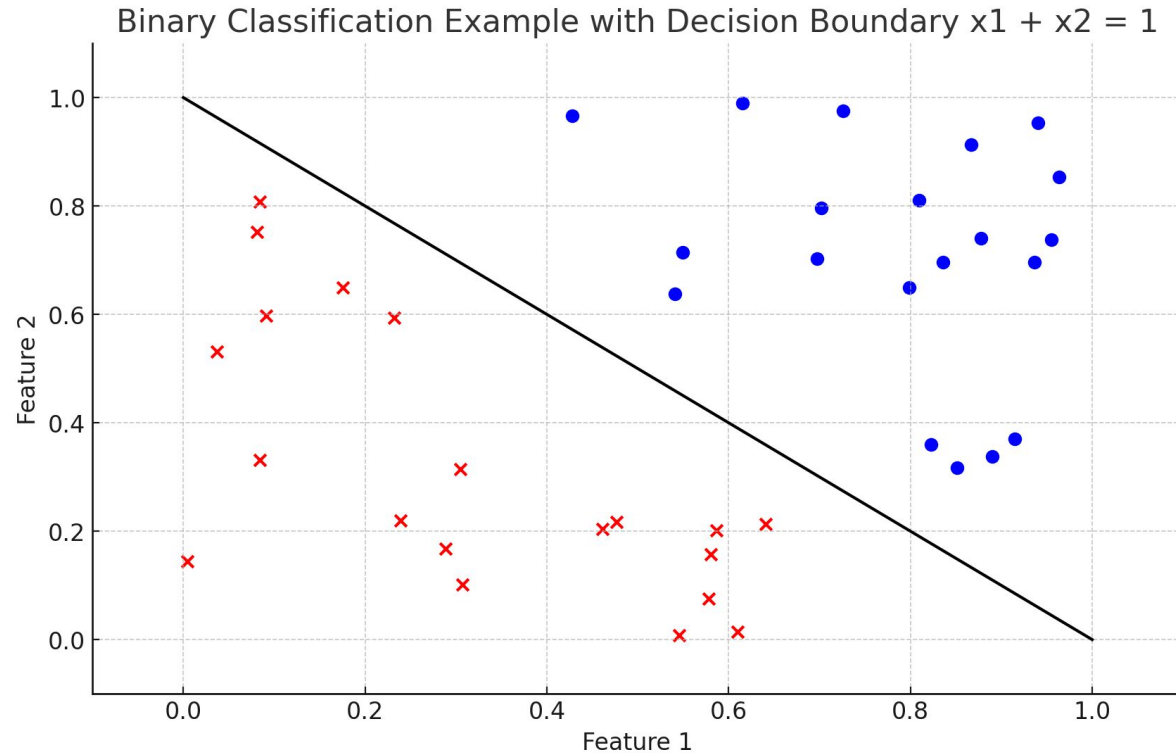
우리 강아지는 귀여워요
uli gang-ajineun gwiyeowoyo

- Regression

Continuous (real) $y \in \mathcal{Y}$



Binary Classification



Binary Classification

- Given dataset (with label)

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots (x^{(n)}, y^{(n)})$$

$$x^{(i)} = (x_1^{(i)}, x_2^{(i)}) \quad y^{(i)} \in \{-1, 1\}$$

- Set function class $\mathcal{G}$
- Set loss function ℓ
- Assumption: Data is linearly separable

Binary Classification

- Linear Classifiers

$$\mathcal{G} = \{g_{a,b}(x) = \text{sign}(a^\top x + b)\}$$

- 0-1 Loss

$$\ell(g_{a,b}(x^{(i)}), y^{(i)}) = \mathbf{1}(g_{a,b}(x^{(i)}) \neq y^{(i)})$$

- Assumption: Linearly Separable

There exists a perfect classifier with Loss=0.

Perceptron Algorithm

Algorithm 8 Perceptron Algorithm for Binary Classification

- 1: Initialize weight vector $\mathbf{a}$ and bias b to zero or small random values
 - 2: Set learning rate η
 - 3: **repeat**
 - 4: **for each** training sample $(\mathbf{x}_i, y_i)$ **do**
 - 5: Compute the weighted sum: $z_i = \mathbf{a} \cdot \mathbf{x}_i + b$
 - 6: Predict the label: $\hat{y}_i = \text{sign}(z_i)$
 - 7: **if** $\hat{y}_i \neq y_i$ **then**
 - 8: Update weights: $\mathbf{a} := \mathbf{a} + \eta(y_i - \hat{y}_i)\mathbf{x}_i$
 - 9: Update bias: $b := b + \eta(y_i - \hat{y}_i)$
 - 10: **end if**
 - 11: **end for**
 - 12: **until** convergence or maximum epochs reached
-

Perceptron

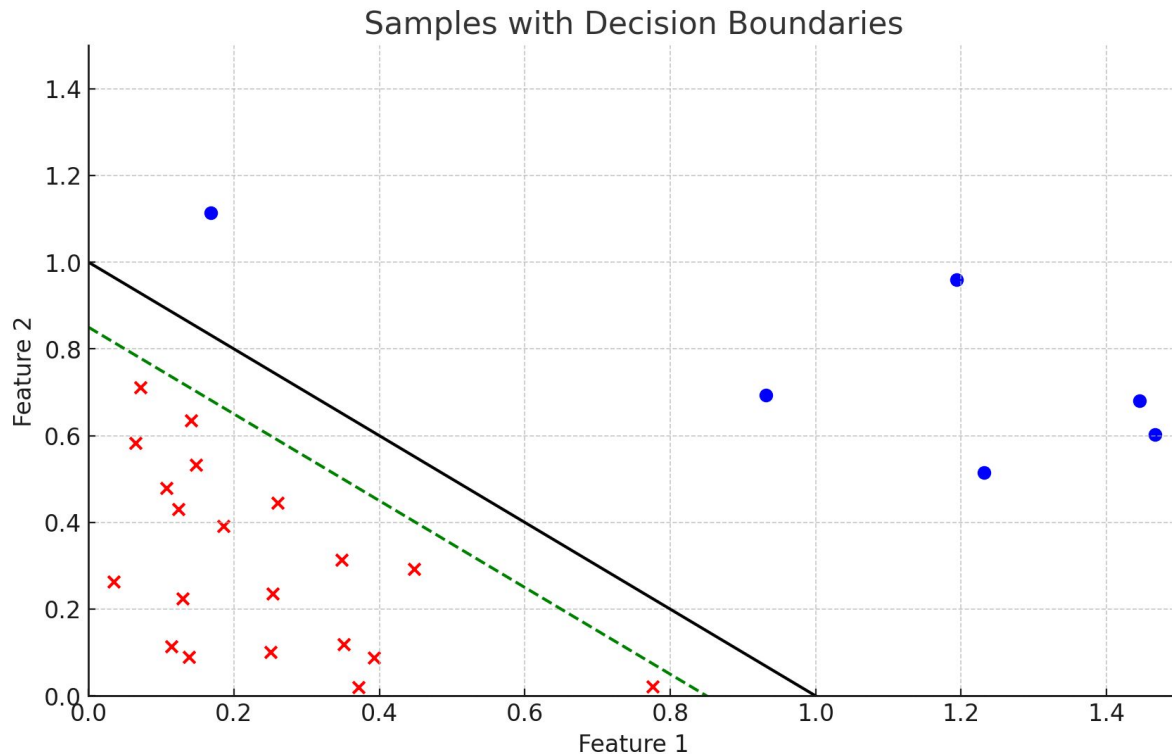
- Rosenblatt (1957)
- Pros:
 - It converges (if there is a solution)
 - It is super simple
- Cons:
 - It never stops if there is no solution
 - You will never know

Linear Programming

$$\begin{array}{ll}\text{minimize} & 0 \\ \text{subject to} & y_i(\mathbf{a} \cdot \mathbf{x}_i + b) > 0 \quad \forall i\end{array}$$

- Null objective
- Only requires to satisfy constraints
- It will tell you if the problem is infeasible

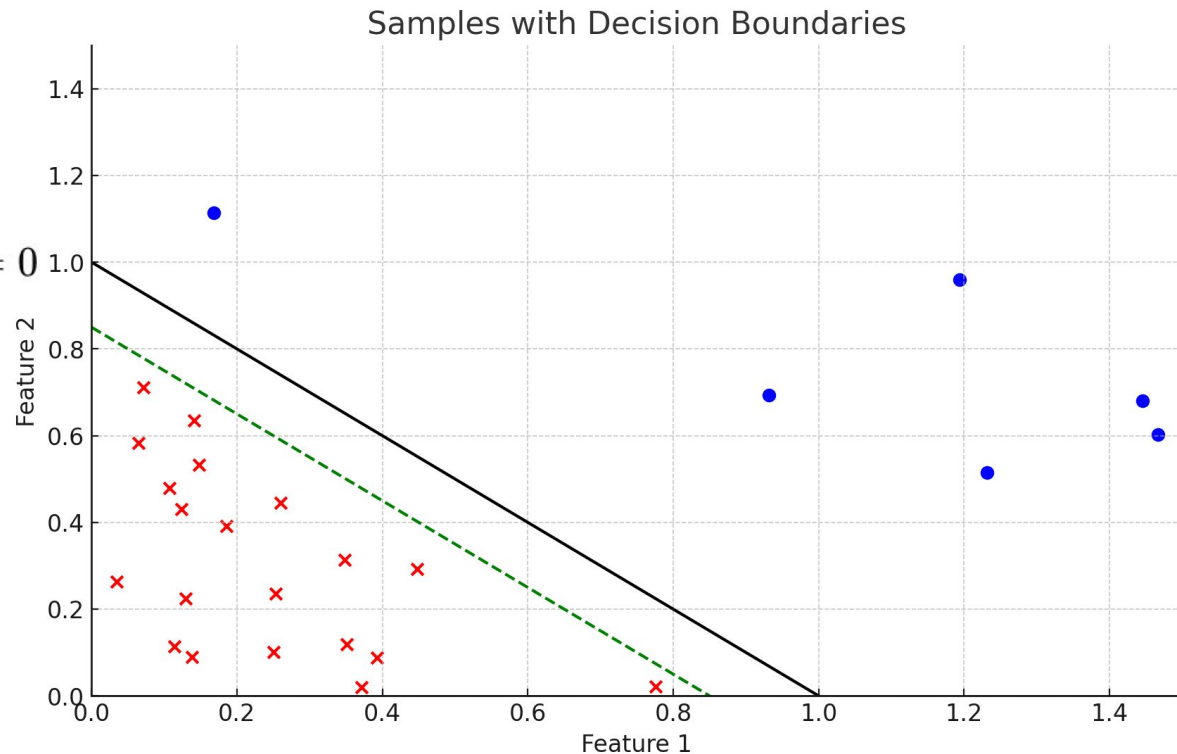
What if we have many solutions?



Margin

- Distance to closest
- Maximum margin
 - Robust
- Distance from $\mathbf{a}^T \mathbf{x} + b = 0$ to a point $\mathbf{x}_0$

$$d = \frac{|\mathbf{a}^T \mathbf{x}_0 + b|}{\|\mathbf{a}\|}$$



Support Vector Machine (SVM)

$$\begin{array}{ll}\text{minimize} & \frac{1}{2} \|\mathbf{a}\|^2 \\ \text{subject to} & y^{(i)} (\mathbf{a} \cdot \mathbf{x}^{(i)} + b) \geq 1 \quad \forall i\end{array}$$

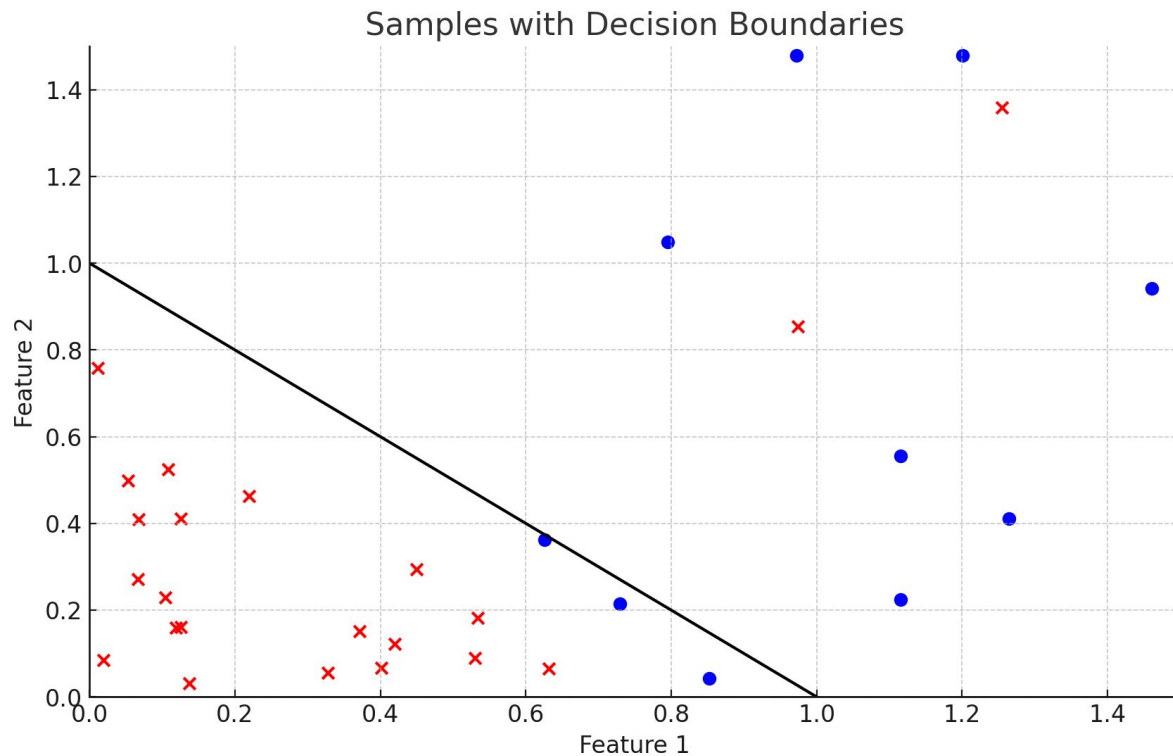
- Fix the constraint, and minimize denominator $d = \frac{|\mathbf{a}^T \mathbf{x}_0 + b|}{\|\mathbf{a}\|}$

SVM

$$\begin{array}{ll}\text{minimize} & \frac{1}{2} \|\mathbf{a}\|^2 \\ \text{subject to} & y^{(i)} (\mathbf{a} \cdot \mathbf{x}^{(i)} + b) \geq 1 \quad \forall i\end{array}$$

- It is convex problem: there are solvers
- Specifically, it is QCQP (Quadratically constrained quadratic program)
- It will tell you if the problem is infeasible

What if we have no solutions?

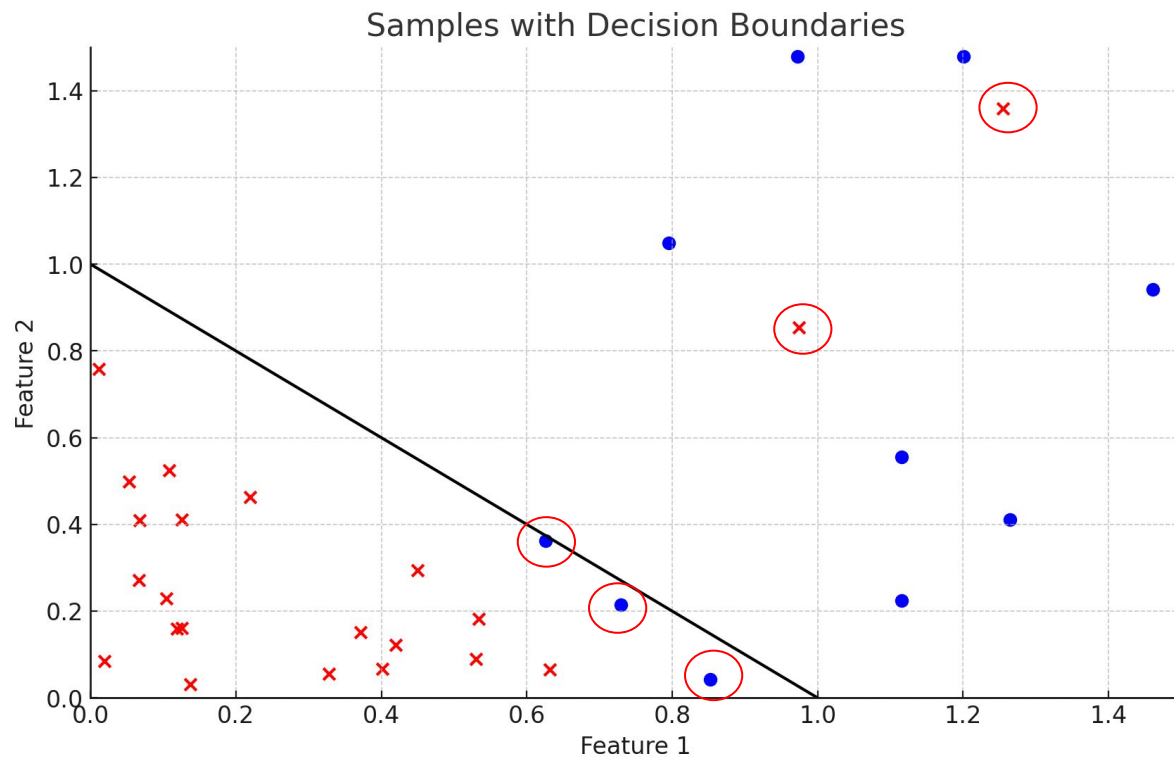


Soft-margin SVM

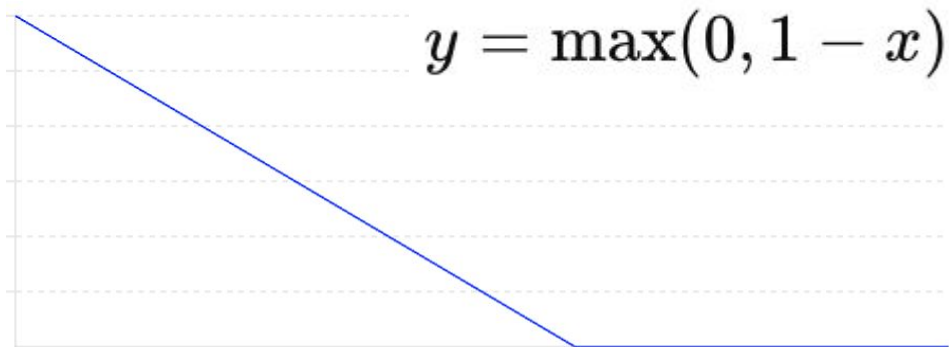
- Current SVM formula does not allow error
(will let you know if it is infeasible)
- Allowing error: ***slack variable (penalty)***

$$\begin{aligned} \text{minimize} \quad & \frac{1}{2} \|\mathbf{a}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{subject to} \quad & y_i(\mathbf{a}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \forall i \\ & \xi_i \geq 0, \quad \forall i \end{aligned}$$

What if we have no solutions?



Hinge Loss



$$\mathcal{L}(y_i, f(\mathbf{x}_i)) = \max(0, 1 - y_i(\mathbf{a}^T \mathbf{x}_i + b))$$

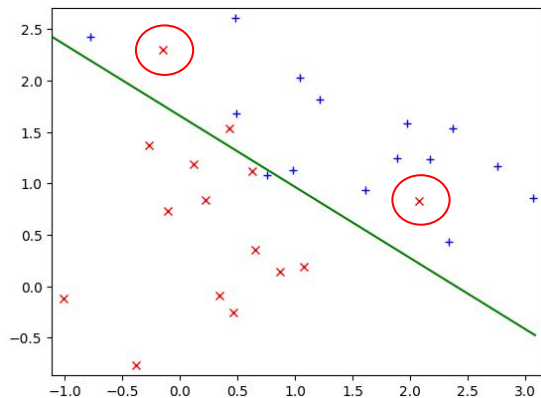
Then, the objective becomes

$$\frac{1}{2} \|\mathbf{a}\|^2 + C \sum_{i=1}^n \mathcal{L}(y_i, f(\mathbf{x}_i))$$

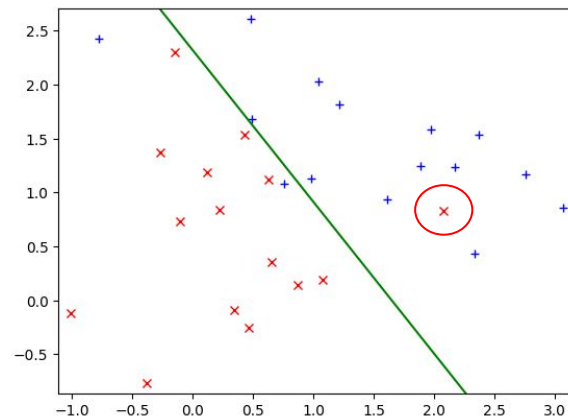
Hinge Loss

$$\frac{1}{2} \|\mathbf{a}\|^2 + C \sum_{i=1}^n \mathcal{L}(y_i, f(\mathbf{x}_i))$$

Larger C : penalize outliers



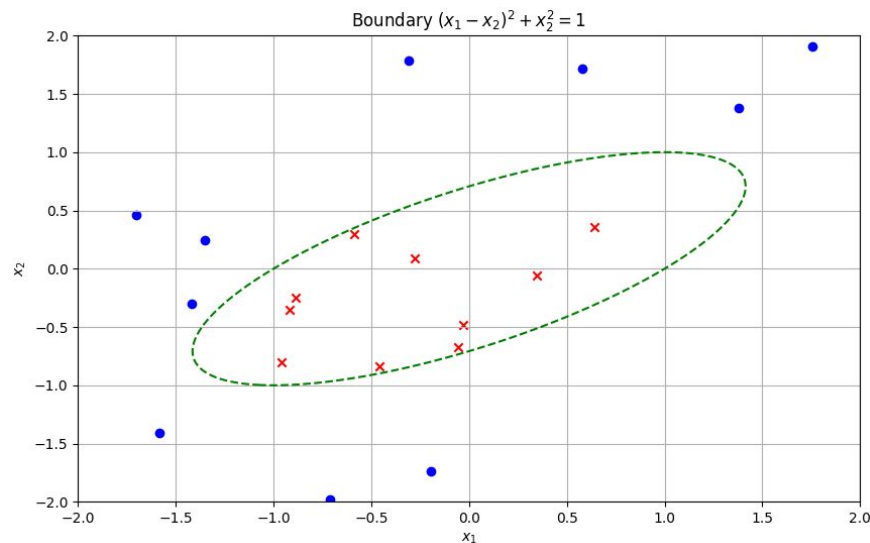
Small C : focus on regular samples



Kernel

- Similar to the Linear Regression, we can allow additional features
- For example, quadratic kernel:

$$\tilde{x} = (x_1, x_2, x_1^2, x_2^2, x_1x_2)$$
$$x_1^2 - 2x_1x_2 + 2x_2^2 = 1$$



Summary: Classification

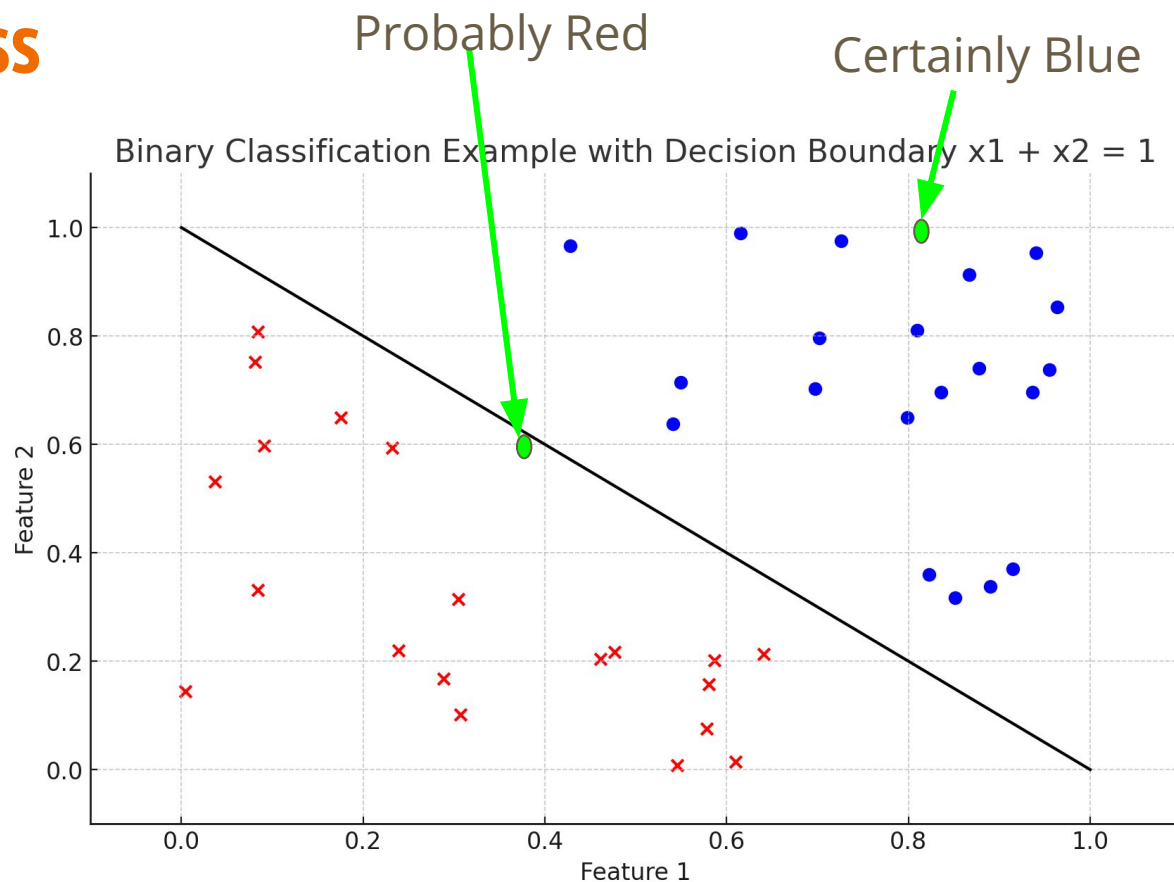
- Perceptron Algorithm
 - 0-1 loss is not differentiable
- SVM
 - 0-1 loss
 - Not differentiable
 - Only considers the points near boundary
- Soft-margin SVM
 - Only considers points near boundary and violation points
 - Requires convex optimization

감사합니다

지도학습 5강

— Logistic Regression —

Soft Guess



Soft Guess

- Hard guess

$$g_{\theta}(x^{(i)}) = 1 \text{ or } -1$$

- Soft guess

$$g_{\theta}(x^{(i)}) = \begin{bmatrix} Pr(y^{(i)} = -1) \\ Pr(y^{(i)} = 1) \end{bmatrix}$$

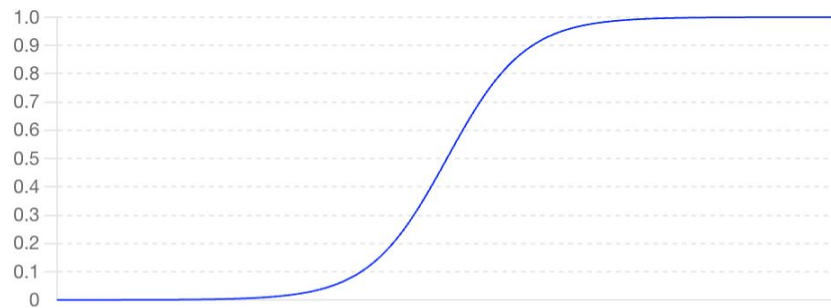
- For example, weather forecast:
it will rain with probability 70% (not rain with 30%)

Logistic Regression: Model Class

- Logistic function:
 - Smooth function
 - Models probability

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

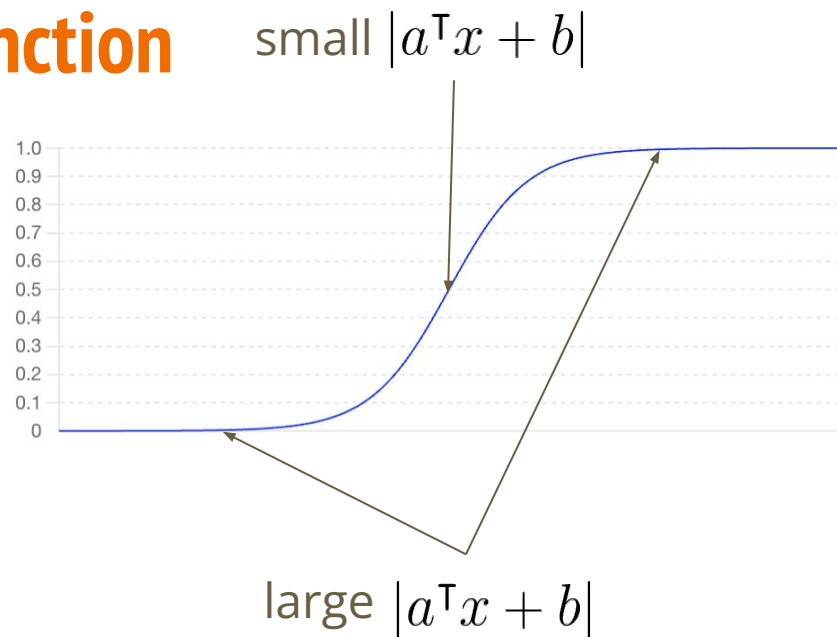
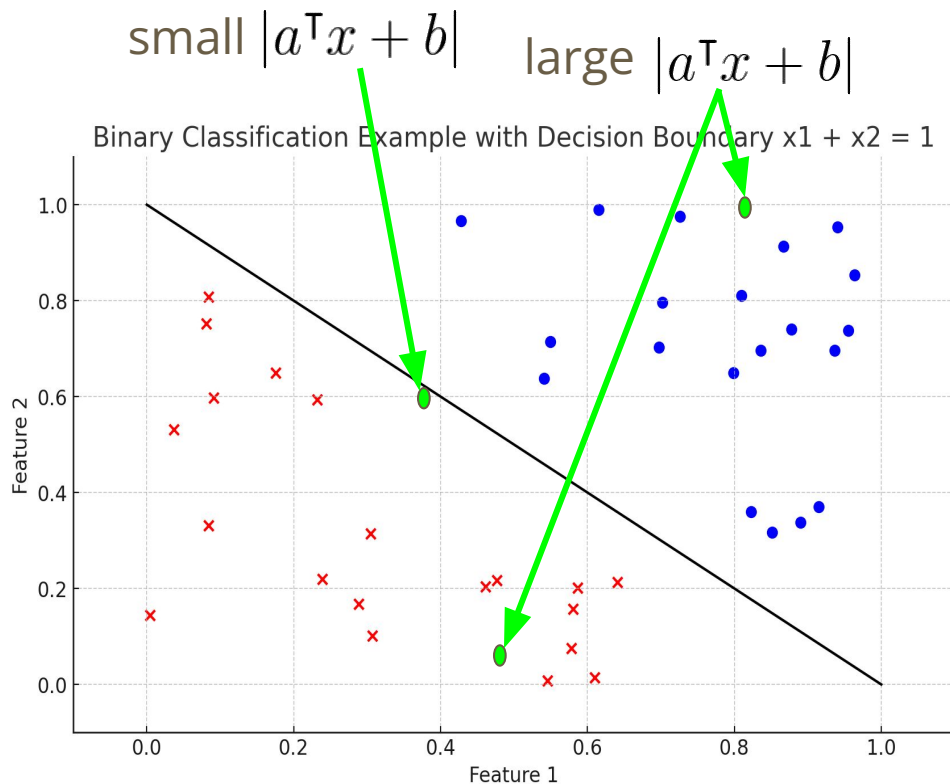
$$g_{a,b}(x) = \begin{bmatrix} 1 - \sigma(a^T x + b) \\ \sigma(a^T x + b) \end{bmatrix}$$



- Model class:

$$\mathcal{G} = \left\{ g_{a,b}(x) = \begin{bmatrix} \frac{e^{-(a^T x + b)}}{1 + e^{-(a^T x + b)}} \\ \frac{1}{1 + e^{-(a^T x + b)}} \end{bmatrix} \right\}$$

Logistic Regression: Logistic Function



$$\mathcal{G} = \left\{ g_{a,b}(x) = \left[\frac{e^{-(a^T x + b)}}{1 + e^{-(a^T x + b)}} \right] \right\}$$

Cross Entropy Loss

- Then, we need to penalize the output $g_{a,b}(x) = \begin{bmatrix} \hat{y}(-1) \\ \hat{y}(1) \end{bmatrix}$
 - If your prediction is 70% raining, and it rains
 - If your prediction is 30% raining, and it rains
- If you assign more probabilities to the event, the loss should be low

$$\ell(g_{a,b}(x^{(i)}), y^{(i)}) = \log \frac{1}{\hat{y}(y^{(i)})}$$

Assigned Probability

True label

Cross Entropy Loss

- Compare $\begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix}$ and $y = 1$, which is simply $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$
- Question, how can we measure “difference” between distributions
- Kullback-Leibler (KL) divergence, aka relative entropy

$$D(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

$$D\left(\begin{bmatrix} 0 \\ 1 \end{bmatrix} \parallel \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix}\right) = 0 \cdot \log \frac{0}{0.7} + 1 \cdot \log \frac{1}{0.3}$$

Logistic Regression

- Dataset (with binary label)

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots (x^{(n)}, y^{(n)})$$

- Function class $\mathcal{G} = \left\{ g_{a,b}(x) = \begin{bmatrix} \frac{e^{-(a^\top x + b)}}{1 + e^{-(a^\top x + b)}} \\ \frac{1}{1 + e^{-(a^\top x + b)}} \end{bmatrix} \right\} \quad g_{a,b}(x) = \begin{bmatrix} \hat{y}(-1) \\ \hat{y}(1) \end{bmatrix}$

- Loss function

$$\ell(g_{a,b}(x^{(i)}), y^{(i)}) = \log \frac{1}{\hat{y}(y^{(i)})}$$

Loss Function

$$Pr(Y = 1) = g_{\theta}(\mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$$

$$Pr(Y = -1) = 1 - g_{\theta}(\mathbf{x})$$

$$x = (\overset{x_0 = 1}{1}, x_1, x_2, \dots, x_d) \in \mathbb{R}^{d+1}$$

$$\mathcal{L}(\theta) = -\frac{1}{n} \sum_{i=1}^n \left[\frac{1 + y^{(i)}}{2} \log(g_{\theta}(\mathbf{x}^{(i)})) + \frac{1 - y^{(i)}}{2} \log(1 - g_{\theta}(\mathbf{x}^{(i)})) \right]$$

$Y=1$

$Y=-1$

$$\log \frac{1}{p} = -\log p$$

Loss Function

$$-\log g_{\theta}(\mathbf{x}) = \log(1 + e^{-\theta^T \mathbf{x}})$$

$$-\log(1 - g_{\theta}(\mathbf{x})) = \log \frac{1 + e^{-\theta^T \mathbf{x}}}{e^{-\theta^T \mathbf{x}}} = \log(1 + e^{\theta^T \mathbf{x}})$$

Thus,

$$-\frac{1 + y^{(i)}}{2} \log(g_{\theta}(\mathbf{x}^{(i)})) - \frac{1 - y^{(i)}}{2} \log(1 - g_{\theta}(\mathbf{x}^{(i)})) = \log(1 + e^{-y^{(i)} \theta^T \mathbf{x}^{(i)}})$$

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \log(1 + e^{-y^{(i)} \theta^T \mathbf{x}^{(i)}})$$

Can apply Gradient Descent!

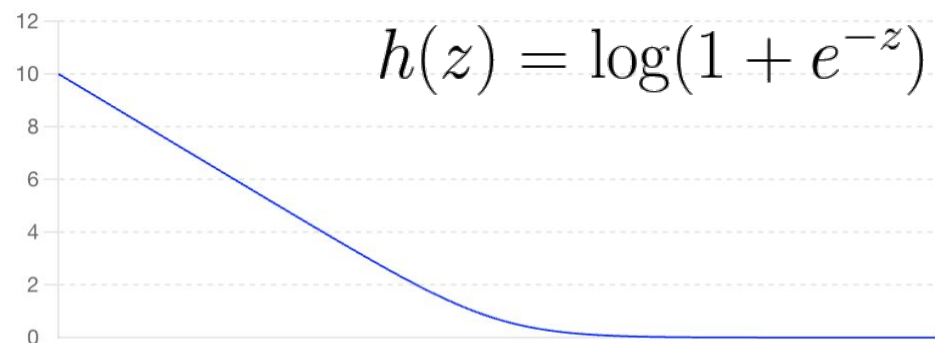
Loss Function

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \log(1 + e^{-y^{(i)} \theta^T \mathbf{x}^{(i)}})$$

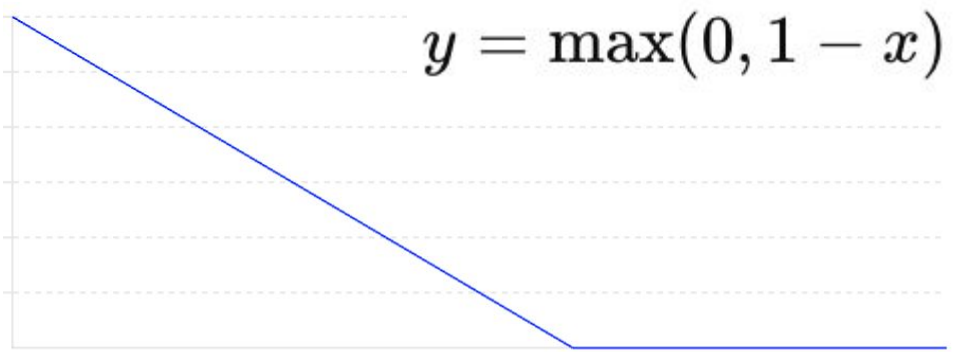
$$h(z) = \log(1 + e^{-z})$$

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n h(y^{(i)} \theta^T \mathbf{x}^{(i)})$$

$$h(z) = \log(1 + e^{-z})$$



Logistic Loss



Hinge Loss

Multiclass Classification

airplane



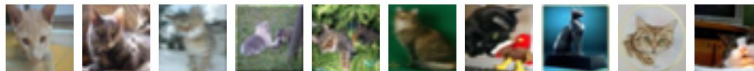
automobile



bird



cat



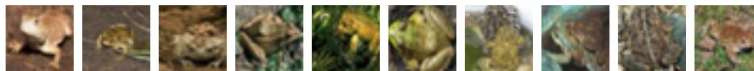
deer



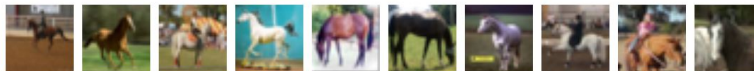
dog



frog



horse



ship

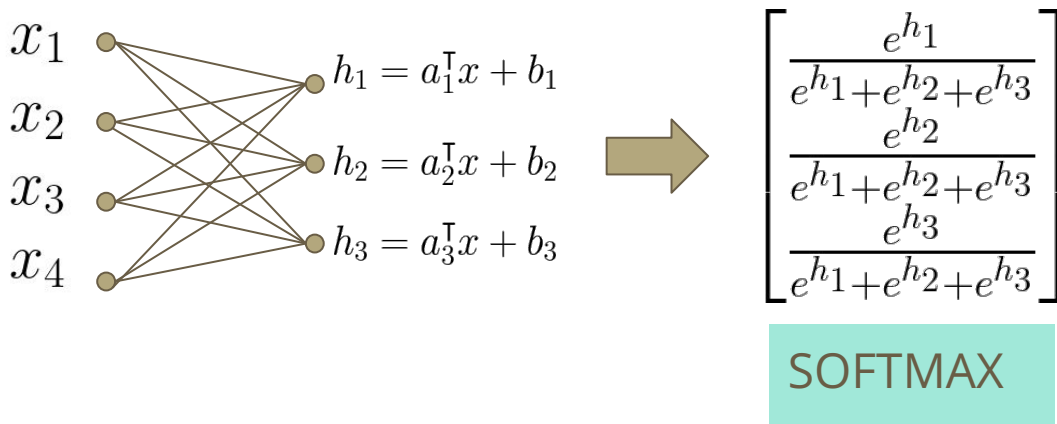


truck



Multiclass Classification

- You can apply multiple binary classification 1 vs. others
- Alternatively,
returns probability distribution on labels.



Softmax Function

- Generalization of logistic function

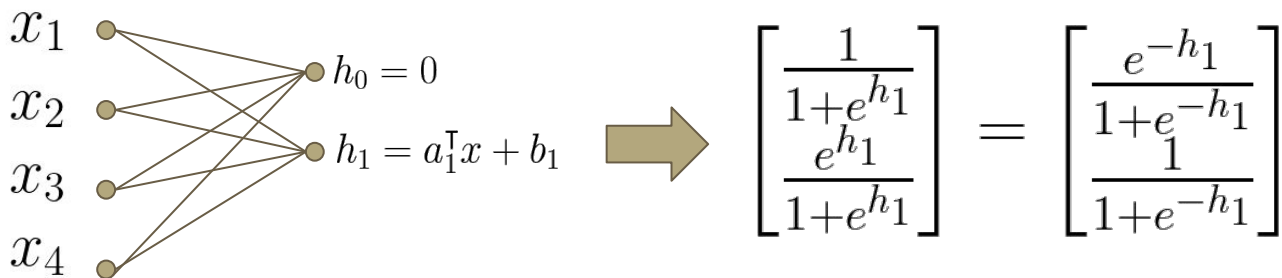
$$\mathbf{z} = [z_1, z_2, \dots, z_K]$$

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

- Output values are positive: exponentiation
- Sum to 1

Logistic Regression $\subseteq$ Softmax Regression

- Binary case requires only one feature (up to normalization)

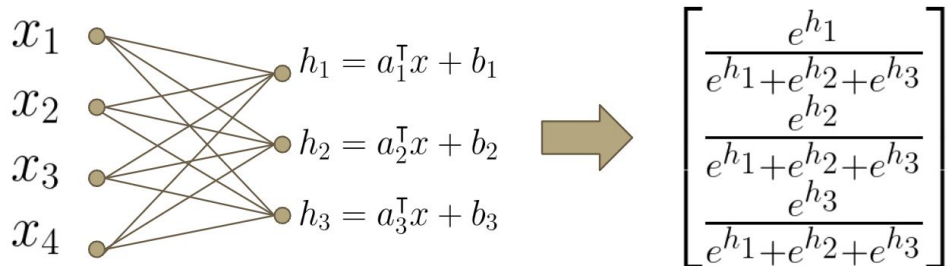


Softmax Regression

- Dataset (with discrete label)

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots (x^{(n)}, y^{(n)})$$

- Function class



- Loss function

$$\ell(g_{a,b}(x^{(i)}), y^{(i)}) = \log \frac{1}{\hat{y}(y^{(i)})}$$

Softmax Regression

- Dataset (with discrete label)

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots (x^{(n)}, y^{(n)})$$

- Function class $h = Ax + B \quad \hat{y} = \sigma(h)$

$$A \in \mathbb{R}^{d \times k}, B \in \mathbb{R}^k$$

- Loss function

$$\ell(g_\theta(x^{(i)}), y^{(i)}) = \log \frac{1}{\hat{y}(y^{(i)})}$$

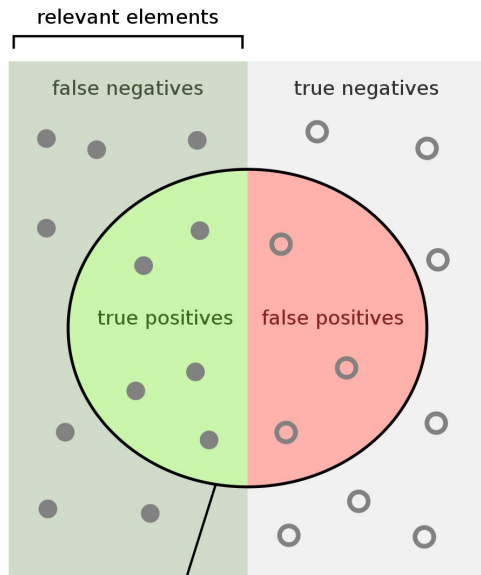
False Positive, False Negative

- If your model outputs $\begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix}$
What would be your final decision?
 - 0.5 would be a safe choice
- But you may want other threshold

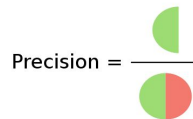
- For example, covid test
 - You may prefer false positive

$$\text{Precision} = \frac{tp}{tp + fp}$$

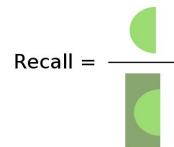
$$\text{Recall} = \frac{tp}{tp + fn}$$



How many retrieved items are relevant?



How many relevant items are retrieved?



Precision and Recall

- Tradeoff: Precision vs. Recall

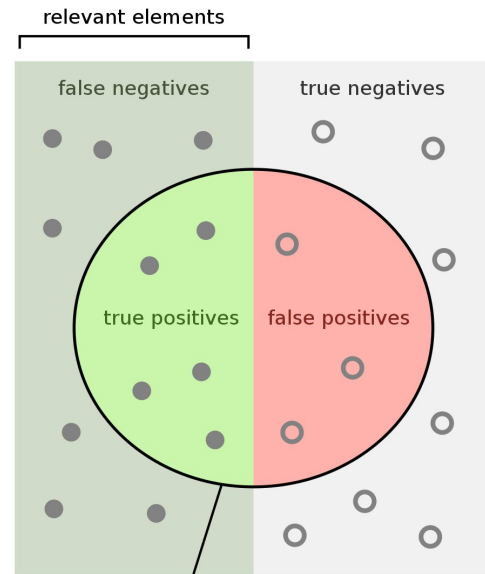
$$\text{Precision} = \frac{tp}{tp + fp}$$

$$\text{Recall} = \frac{tp}{tp + fn}$$

- Based on your application

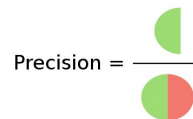
- **F1-score**

Harmonic mean of Precision and Recall

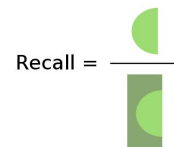


retrieved elements

How many retrieved items are relevant?

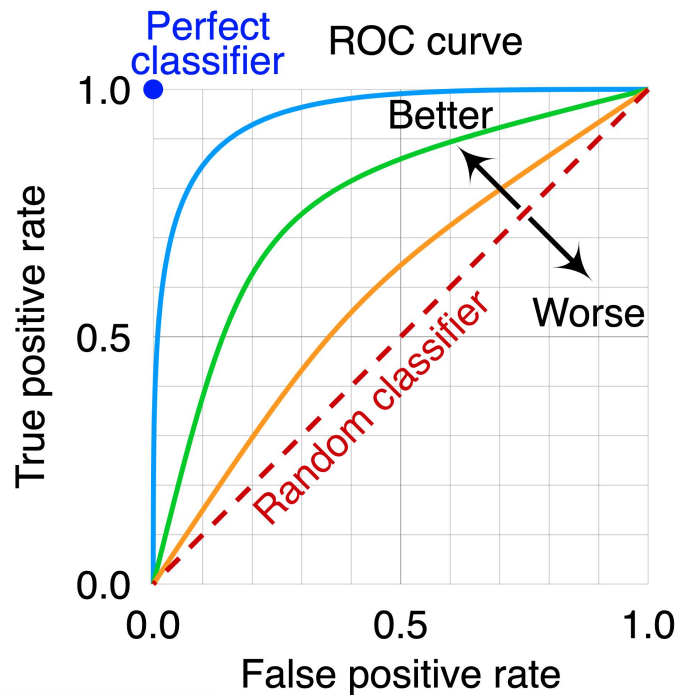


How many relevant items are retrieved?



Precision, Recall, and ROC curve

- **True positive rate (TPR)**
 $TP/(TP+FN)$
- **False positive rate (FPR)**
 $FP/(FP+TN)$
- **Always say positive**
 $FN=TN=0, TPR=FPR=1$
- **Always say negative**
 $TP=FP=0, TPR=FPR=0$
- **Random Guess**
No difference between TPR, FPR



Summary: Logistic Regression

- Soft Guess
- Cross Entropy Loss
- Softmax Classification
- Prediction and Recall

감사합니다

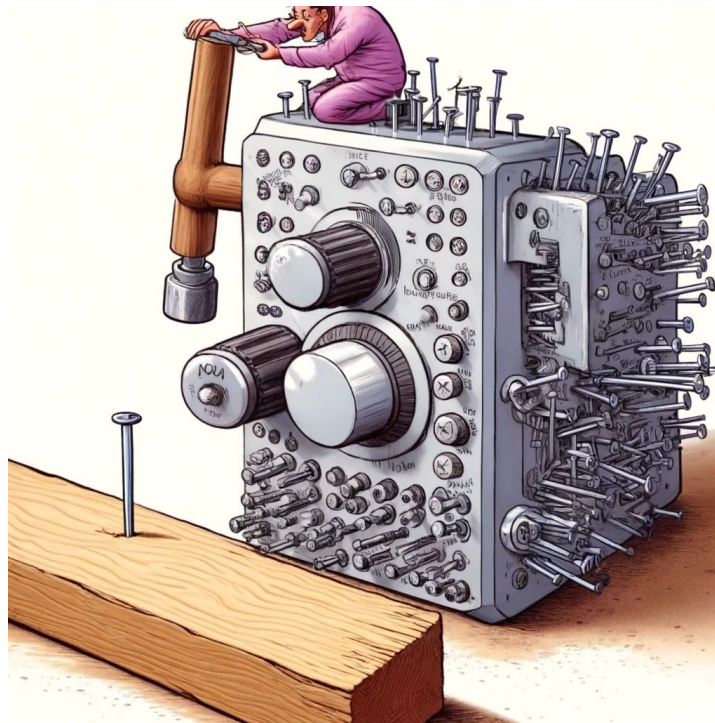
지도학습 6강

— More on Supervised Learning
and Beyond —

You might not need deep learning

- If your problem is simple then your model should be simple
- Deep learning might be overkill

- Image generated by ChatGPT 4o



Naive Bayes: Spam Filter

- You can have explicit (interpretable) model
- Represent email with indicator vector
 - Whether it contains specific words
- *Naive Bayes Assumption*
 - X's are independent given y

$$\begin{aligned} & p(x_1, \dots, x_{50000} | y) \\ &= p(x_1 | y) p(x_2 | y, x_1) p(x_3 | y, x_1, x_2) \cdots p(x_{50000} | y, x_1, \dots, x_{49999}) \\ &= p(x_1 | y) p(x_2 | y) p(x_3 | y) \cdots p(x_{50000} | y) \\ &= \prod_{j=1}^d p(x_j | y) \end{aligned}$$

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{array}{l} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{array}$$

Naive Bayes: Spam Filter

$$\phi_{j|y=1} = p(x_j = 1|y = 1)$$

$$\phi_{j|y=0} = p(x_j = 1|y = 0)$$

$$\phi_y = p(y = 1)$$

$$\phi_{j|y=1} = \frac{\sum_{i=1}^n 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_{i=1}^n 1\{y^{(i)} = 1\}}$$

$$\phi_{j|y=0} = \frac{\sum_{i=1}^n 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 0\}}{\sum_{i=1}^n 1\{y^{(i)} = 0\}}$$

$$\phi_y = \frac{\sum_{i=1}^n 1\{y^{(i)} = 1\}}{n}$$

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{array}{l} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{array}$$

Naive Bayes: Spam Filter

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{matrix} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{matrix}$$

$$\begin{aligned} p(y = 1|x) &= \frac{p(x|y = 1)p(y = 1)}{p(x)} \\ &= \frac{\left(\prod_{j=1}^d p(x_j|y = 1)\right) p(y = 1)}{\left(\prod_{j=1}^d p(x_j|y = 1)\right) p(y = 1) + \left(\prod_{j=1}^d p(x_j|y = 0)\right) p(y = 0)} \end{aligned}$$

Naive Bayes: Laplace Smoothing

There might be a word that never appeared

$$\phi_{35000|y=1} = \frac{\sum_{i=1}^n 1\{x_{35000}^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_{i=1}^n 1\{y^{(i)} = 1\}} = 0$$

$$\phi_{35000|y=0} = \frac{\sum_{i=1}^n 1\{x_{35000}^{(i)} = 1 \wedge y^{(i)} = 0\}}{\sum_{i=1}^n 1\{y^{(i)} = 0\}} = 0$$

Naive Bayes: Laplace Smoothing

Start count with 1

$$\phi_{j|y=1} = \frac{1 + \sum_{i=1}^n 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\}}{2 + \sum_{i=1}^n 1\{y^{(i)} = 1\}}$$

$$\phi_{j|y=0} = \frac{1 + \sum_{i=1}^n 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 0\}}{2 + \sum_{i=1}^n 1\{y^{(i)} = 0\}}$$

Decision Tree

- flowchart-like structure
- Internal node: represents a decision
- Branch: outcome of that decision
- Leaf node
 - Class label (classification)
 - Value (regression)

Decision Tree: Classification

- *animal is a mammal or not*
- Root Node: The first decision is based on whether the animal "Has Fur."
 - If "Yes," go to the next decision.
 - If "No," classify as "Not a Mammal."
- Next Decision: For animals that "Have Fur," the next decision is based on whether the animal "Gives Birth."
 - If "Yes," classify as "Mammal."
 - If "No," classify as "Not a Mammal."



Decision Tree: Regression

- *predicting the price of a house based on two features:*
 - *"Size in Square Feet" and "Number of Bedrooms."*
- Root Node: The first decision is based on whether the "Size in Square Feet" is greater than 2000.
 - If "Yes," go to the next decision.
 - If "No," the price is \$150,000.
- Next Decision: For houses larger than 2000 square feet, the next decision is based on whether the "Number of Bedrooms" is greater than 3.
 - If "Yes," the price is \$300,000.
 - If "No," the price is \$250,000.



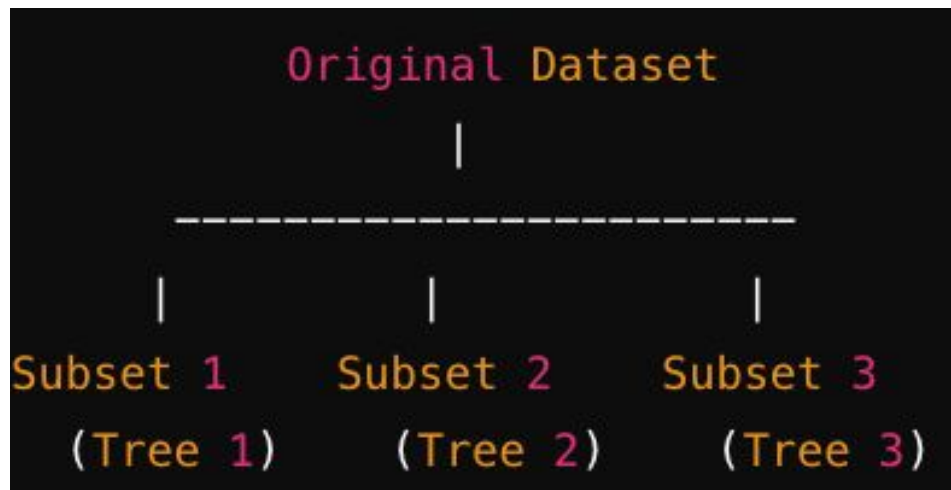
Decision Tree

- *Splitting Criteria*
- Classification
 - Measure the “dispersion” (entropy)
- Regression
 - Measure the variance within the subsets



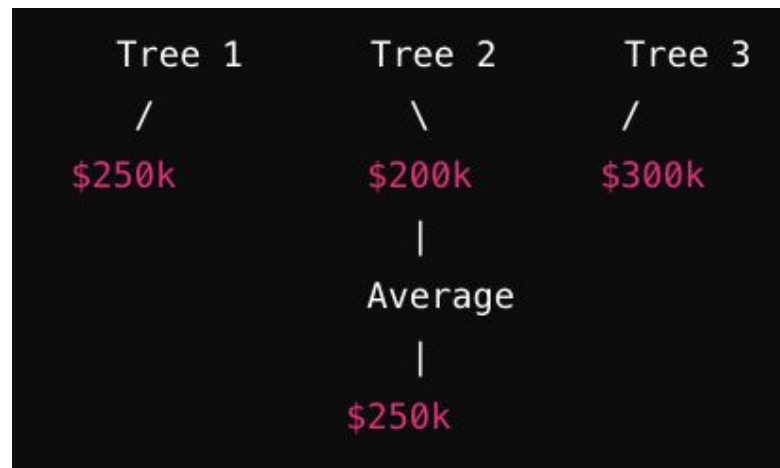
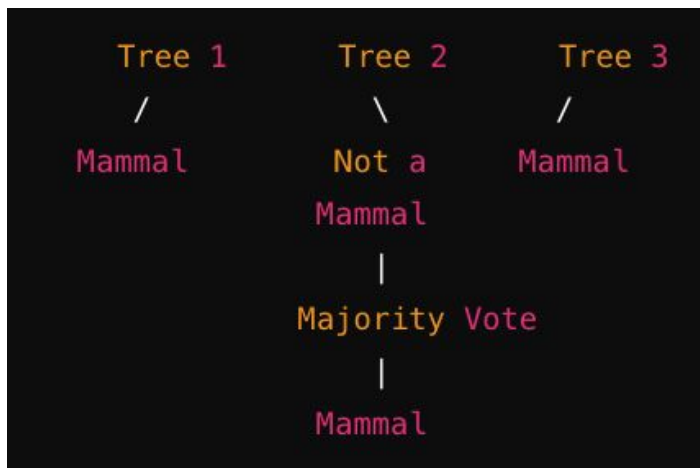
Bagging

- Bootstrap Aggregating
- multiple subsets of the training data
 - sampling with replacement.
- Reduces overfitting
- Improves model robustness



Random Forest

- Ensemble method
- Combines multiple decision trees



Adaptive Boosting (AdaBoost)

- Ensemble method
 - Sequential Learning
 - Models sequentially correct errors
-
- Train the first model, then weigh more on misclassified samples
 - Train the next model with weights
 - Finally, weighted sum of predictions (by models)
 - weights : based on error rate

Bagging vs. Boosting

Bagging

- Parallel
- Reduce variance
- Aggregation
 - voting (classification)
 - Averaging (regression)

Boosting

- Sequential
- Reduce bias
- Aggregation
 - Weighted majority voting
 - Weighted averaging

Other Supervised Learning Examples

Supervised Learning Examples: Super Resolution

bicubic
(21.59dB/0.6423)



SRResNet
(23.53dB/0.7832)



SRGAN
(21.15dB/0.6868)



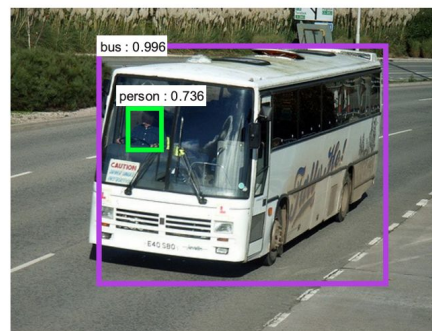
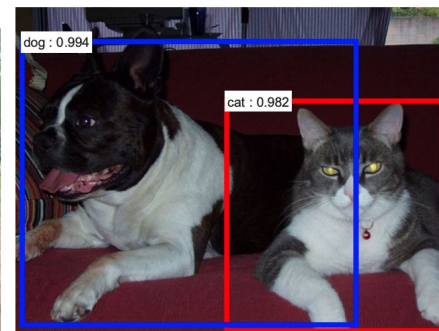
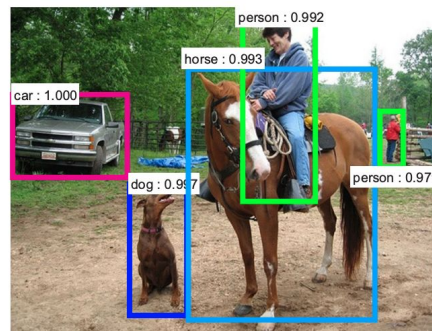
original



- Data: (low resolution, high resolution)
- Model: Deep learning (CNN)
- Loss: Pixel-wise MSE or PSNR

Supervised Learning Examples: Super Resolution

- Data: (image, box and label)
- Model: Deep learning (CNN)
- Loss: difference (box coordinate) + classification loss



Supervised Learning Examples: BERT

- Data: (Masked sentence, tokenized sentence)
- Model: Transformer (LLM)
- Loss: Classification Loss

Original Sentence:	The quick brown fox jumps over the lazy dog
Tokenized Sentence:	[The] [quick] [brown] [fox] [jumps] [over] [the] [lazy] [dog]
Masked Sentence:	[The] [quick] [MASK] [fox] [jumps] [over] [MASK] [lazy] [dog]
Input to BERT:	[The] [quick] [MASK] [fox] [jumps] [over] [MASK] [lazy] [dog]

Supervised Learning Examples: BERT

- Data: (pair of sentence, binary (whether they are consecutive))
- Model: Transformer (LLM)
- Loss: Classification Loss

Original Sentences:

Sentence A: "The quick brown fox jumps over the lazy dog."

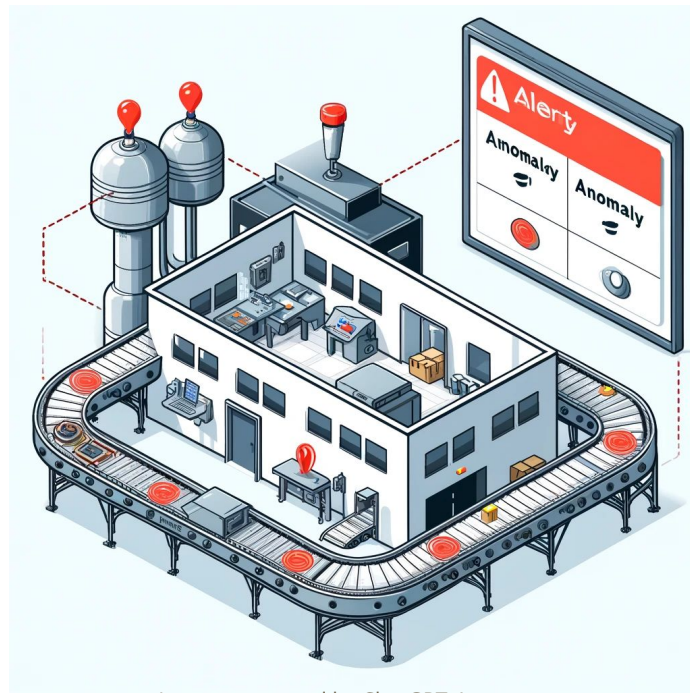
Sentence B: "The lazy dog did not see the fox coming."

NSP Prediction:

Predicted Next Sentence: Yes

Supervised Learning Examples: Anomaly Detection

- Data: (sensory inputs, anomaly indicator)
- Model: Deep Learning Models
- Loss: (weighted) Classification Loss

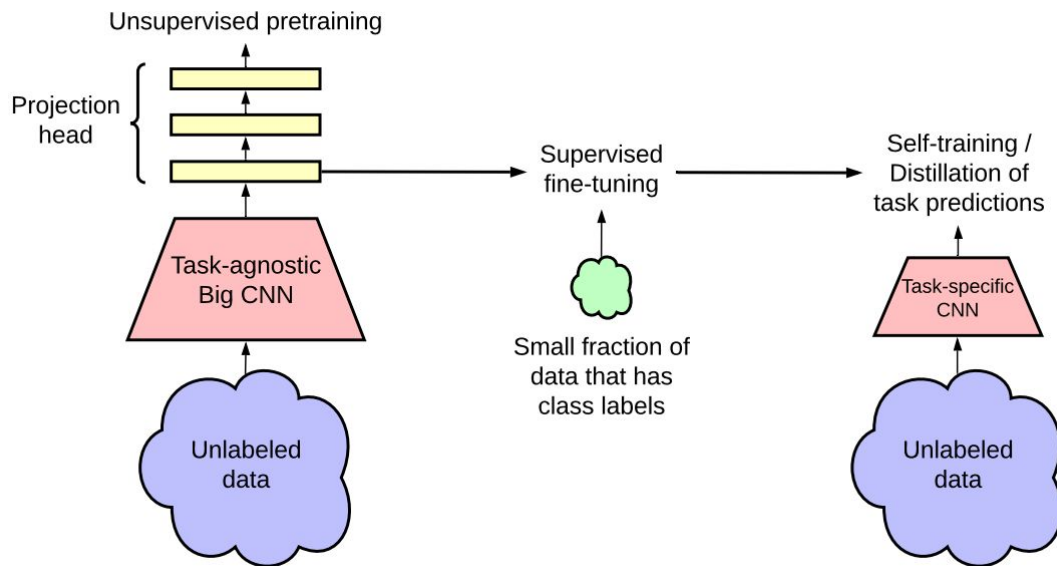


- Image generated by ChatGPT 4o

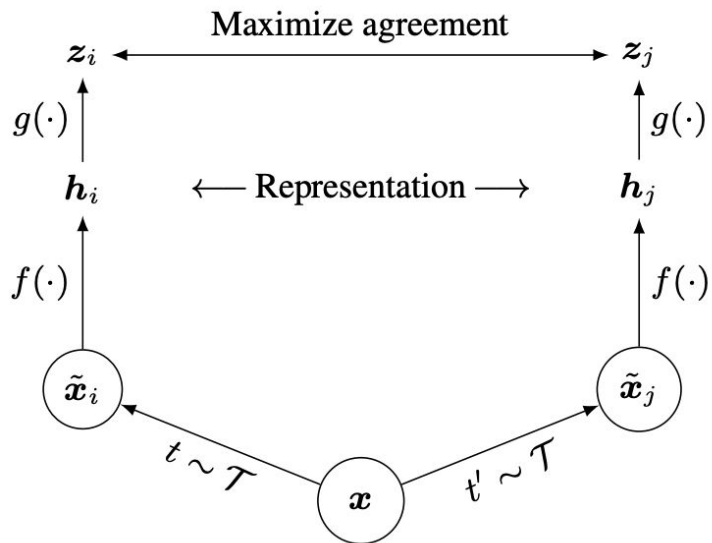
Beyond Supervised Learning

Semi-Supervised Learning

- Labeling is expensive
- Learn from unlabeled data (augmentation)



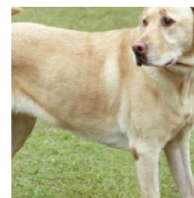
Semi-Supervised Learning



(a) Original



(b) Crop and resize



(c) Crop, resize (and flip)



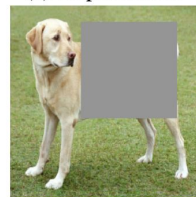
(d) Color distort. (drop)



(e) Color distort. (jitter)



(f) Rotate $\{90^\circ, 180^\circ, 270^\circ\}$



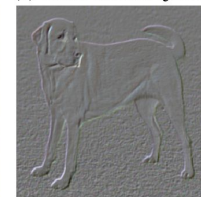
(g) Cutout



(h) Gaussian noise



(i) Gaussian blur



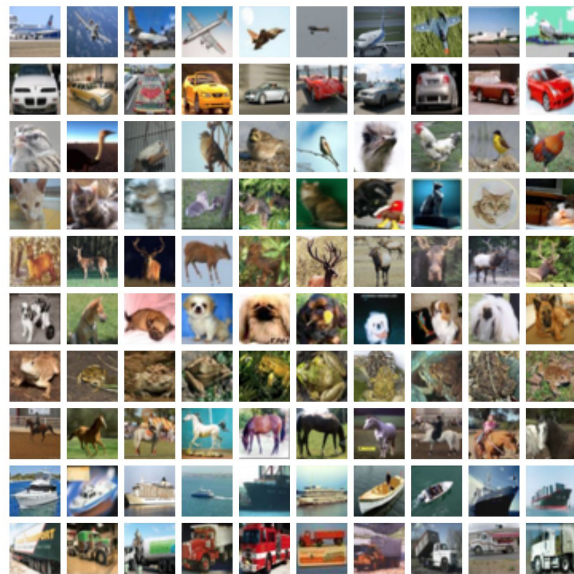
(j) Sobel filtering

Generative Models

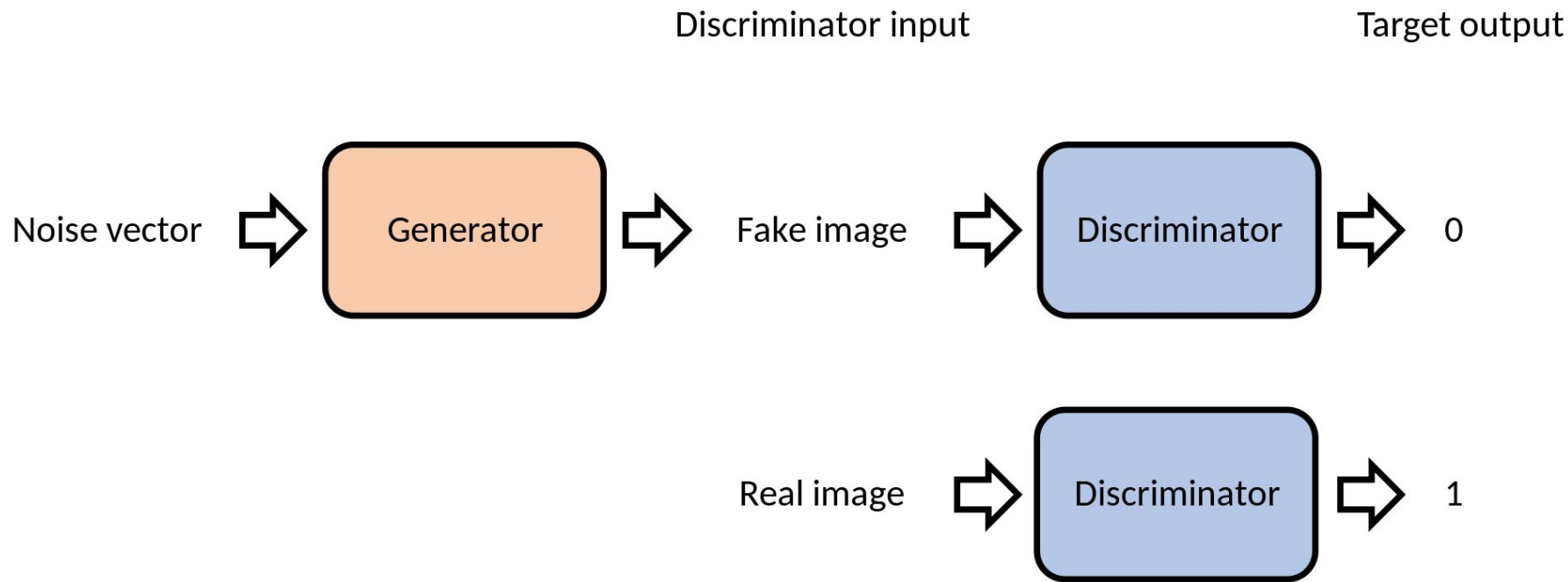
- Does not have label: only have x 's

Goal

- Learn the distribution of data $p(x)$



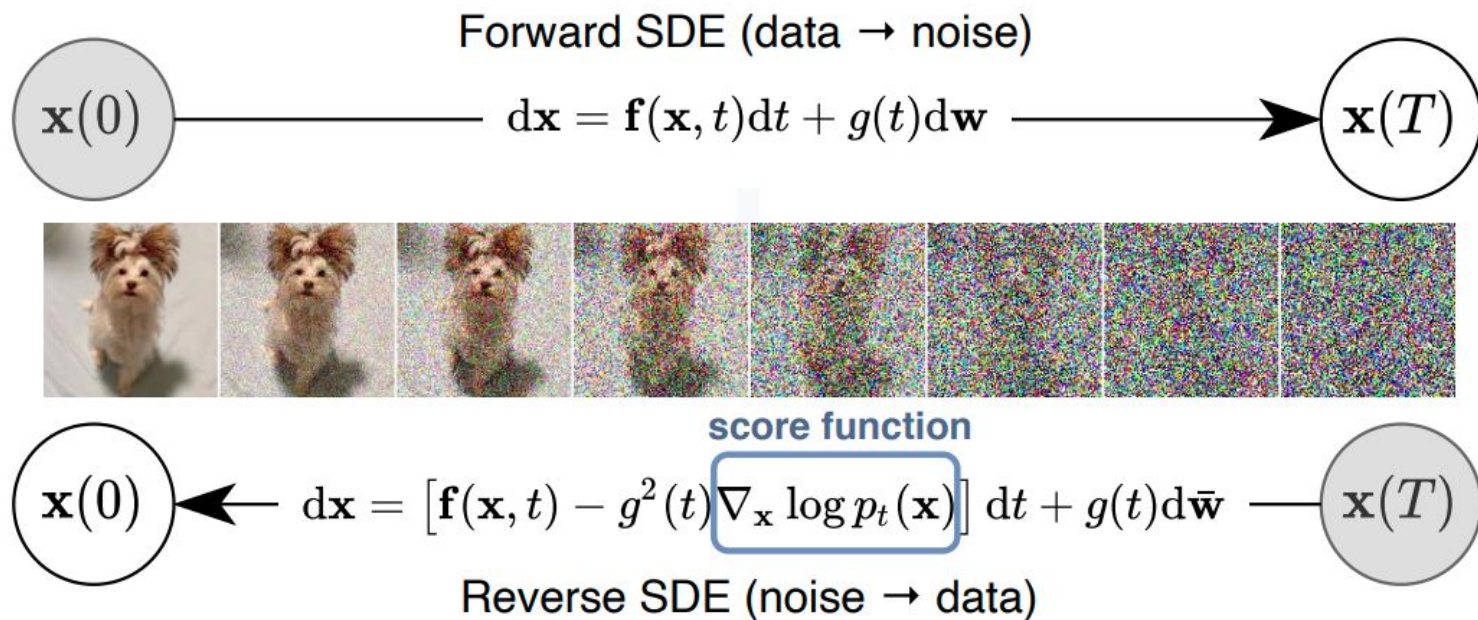
Generative Adversarial Networks



Diffusion Models



Diffusion Models



감사합니다